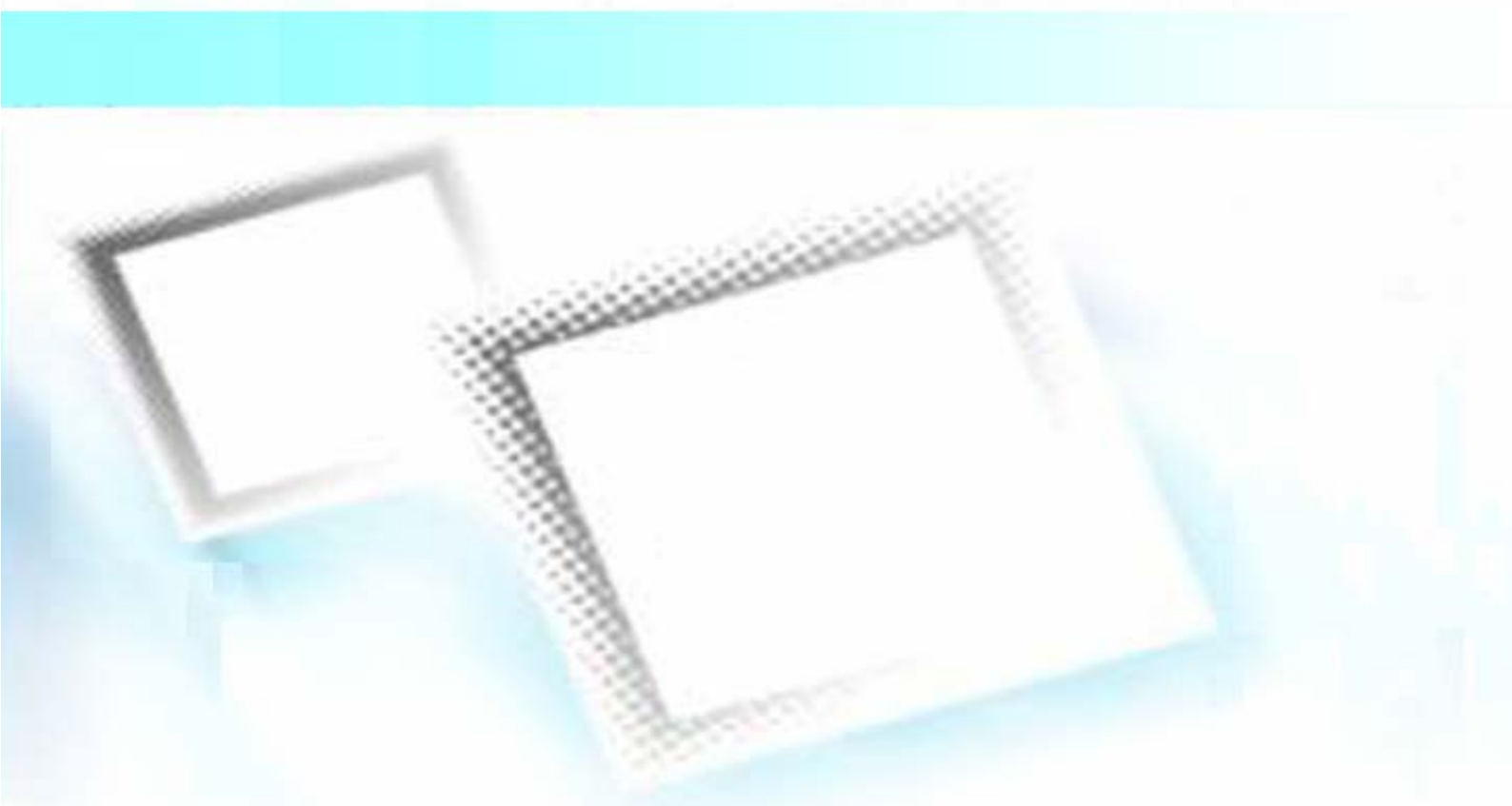




盛科 V350/V150 用户指导

版本	R1.6
日期	2016-01-21

版权所有 © 盛科网络（苏州）有限公司。保留一切权利。

未经盛科网络（苏州）有限公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式和任何方法传播。



盛科商标，服务标志和其他盛科标志均为盛科网络（苏州）有限公司拥有商标。盛科交换机系列产品和芯片系列产品的标志均为盛科网络（苏州）有限公司商标或注册商标。未经盛科书面授权，不允许使用这些标志。本文档提及的其他所有商标和商业名称，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受盛科网络商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，本公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

盛科网络（苏州）有限公司

地址 江苏省苏州市工业园区星汉街5号（腾飞新苏工业坊）B幢4楼13/16单元

电话 86-512-62885358

传真 86-512-62885870

网址 <http://www.centecnetworks.com>

邮箱 support@centecnetworks.com

目录

1 前言	8
1.1 更新记录	8
2 流表模式	9
2.1 概述	9
2.2 配置流表模式	9
2.3 默认流表模式规格	10
3 配置流表	12
3.1 概述	12
3.2 示例拓扑	12
3.3 配置转发到单个端口的流表	13
3.4 配置转发到逻辑端口 ALL 的流表	13
3.5 配置转发到控制器的流表	14
3.6 配置丢弃报文的流表	14
3.7 配置修改二层字段并转发到多个端口的流表	15
3.8 配置修改三层字段并转发到多个端口的流表	16
3.9 配置修改四层字段并转发到多个端口的流表	16
3.10 配置带优先级和超时的流表	17
3.11 修改流表的行为	18
3.12 删除指定的流表	19
3.13 删除所有流表	20
3.14 查询单条流表统计	21
3.15 查询流表全局统计	22
4 配置 MPLS	24
4.1 概述	24
4.2 参考资料	24
4.3 配置 IP over MPLS	25
4.4 配置 VPWS	27
4.5 配置 VPLS	30
4.6 OpenFlow Spec 1.3 修改	32
4.7 基于 Nicira 的扩展流表行为	33
4.8 MPLS 支持的 action	34

5 配置 GRE 隧道	36
5.1 概述	36
5.2 配置 GRE 隧道	36
5.3 配置 GRE 隧道 id	39
5.4 Update GRE tunnel nexthop MAC	41
6 配置控制器	45
6.1 概述	45
6.2 示例拓朴	45
6.3 控制器配置	46
6.3.1 OpenDaylight	46
6.3.2 ryu	48
6.3.3 floodlight	49
6.4 交换机控制器配置	51
7 配置 Meter	52
7.1 概述	52
7.2 参考资料	52
7.3 配置 Meter	52
8 配置 QinQ	54
8.1 概述	54
8.2 参考资料	54
8.3 配置 QinQ	54
9 配置 Group	56
9.1 概述	56
9.2 参考资料	56
9.3 配置 ALL 类型 Group	57
9.4 配置 SELECT 类型 Group	58
9.5 配置 INDIRECT 类型 Group	59
9.6 配置 FAST FAILOVER 类型 Group	60
10 配置 QoS	62
10.1 概述	62
10.2 参考资料	62
10.3 配置 Queue 整形	62
10.4 配置 Classification	63
10.5 配置 Scheduling	66
11 配置 sFlow	69
11.1 概述	69
11.2 参考资料	69

11.3 配置 sFlows.....	69
12 配置 Bond.....	72
12.1 概述.....	72
12.2 配置 Bond	72
12.3 配置 Bond Hash 参数	74
13 配置二层 hash 模式.....	75
13.1 概述.....	75
13.2 参考资料.....	76
13.3 配置二层 hash 流表模式.....	76
13.4 配置二层 hash key.....	76
13.5 配置二层 hash 流表.....	77
14 配置三层 hash 流表模式.....	78
14.1 概述.....	78
14.2 参考资料.....	79
14.3 配置 l3hash 流表模式	79
14.4 配置三层 hash key.....	79
14.5 配置三层 hash 流表.....	80
15 配置 NAT 模式	82
15.1 概述.....	82
15.2 参考资料.....	83
15.3 配置 NAT 流表模式	84
15.4 在 5 元组子模式下配置流表	84
15.5 配置 DNAT 流表的负载分担	85
16 配置 NVCloud 模式.....	87
16.1 概述.....	87
16.2 参考资料.....	88
16.3 配置 nvcloud 流表模式	88
16.4 配置 nvgre compatible	89
16.5 配置租户识别类型	89
16.6 配置 L2 forward flow	91
16.7 配置 L3 forward flow	94
17 配置 Multi-table 模式.....	98
17.1 概述.....	98
17.2 参考资料.....	100
17.3 配置 multi-table 流表模式	100
17.4 配置 multi-table 的 wildcard match 流表.....	100
17.5 配置 multi-table 的 tunnel 流表	101

18 配置 IPv6 模式	103
18.1 概述.....	103
18.2 参考资料.....	104
18.3 Configure ipv6 profile.....	104
18.4 Add ipv6 wildcard match flows.....	105
19 配置 LACP.....	106
19.1 概述.....	106
19.2 配置 LACP.....	106
19.3 配置 LACP 参数.....	108
20 配置启动时加载 flows/groups/meters	110
20.1 概述.....	110
20.2 配置启动时加载 flows.....	110
20.3 配置启动时加载 groups.....	111
20.4 配置启动时加载 meters	111
21 附录.....	113
21.1 附录 A.....	113
21.2 附录 B.....	118
A.1.1 对 SSL 配置的支持.....	121

附图

Figure 3-1 : OpenFlow 配置示例拓扑	12
Figure 4-1 : IP over MPLS 拓扑	25
Figure 4-2 : VPWS 拓扑	28
Figure 4-3 : VPLS 拓扑	30
Figure 5-1 : GRE Tunnel Topology	37
Figure 5-2 : GRE Tunnel Topology	39
Figure 5-3 : GRE 隧道拓扑	41
Figure 6-1 : 采用一个控制器 OpenFlow 网络	45
Figure 6-2 : 采用三个控制器 OpenFlow 网络	46
Figure 7-1 : Meter Topology	52
Figure 8-1 : QinQ 拓扑	55
Figure 11-1 : sFlow 拓扑	70
Figure 12-1 : Bond port 拓扑	72
Figure 19-1 : Lacp 拓扑	106

1 前言

1.1 更新记录

日期	镜像版本	描述
2015-02-03	V3.1.16.1.fcs	中文文档初版
2015-07-13	V3.1.16.3.fcs	修复 L3HASH 流表模式能力
2016-01-21	V3.1.16.4.fcs	修复 nat profile 问题

2 流表模式

2.1 概述

借助于盛科第三代高性能芯片 CTC5163 的 FTM（Flexible Table Management）技术，针对特定的应用场景，V350/V150 提供了灵活的流表模式，从而在特定的应用场景上，最大化流表规格。

通过对当前 OpenFlow 应用场景的研究，V350/V150 目前支持以下流表模式配置：

1. 默认模式：拥有 2000 条 wildcard 流表，支持 OpenFlow Meter、Group 等高级特性，适用于对灵活性要求较高但流表规模不大的部署场景。
2. 二层 hash 流表模式：拥有 16000 条精确匹配二层流表，适用于需要大量二层流表的场景。
3. 三层 hash 流表模式：拥有 32000 条精确匹配三层流表，适用于需要大量三层流表的场景。
4. NAT 模式：可支持 2000 条 wildcard 流表。
5. 多级流表模式：Table0 支持 500 条 wildcard 流表和 6000 条 tunnel 流表，Table1 支持 1000 条 wildcard 流表。适用于需要两级流表的场景。
6. IPv6 模式：支持 500 条 wildcard 流表，适用于需要匹配和编辑 IPv6 的场景。

盛科将继续研究 OpenFlow 领域的最新应用场景，并提供多样化的流表模式，以满足不同用户的需求。

2.2 配置流表模式

在如下配置示例中，将 V350/V150 切换到默认流表模式。

Switch(config)# stm prefer default	切换到默认流表模式
------------------------------------	-----------



新模式需要保存配置重启后生效。

2.3 默认流表模式规格

流表数量	2000 条 wildcard 流表
Wildcard 匹配字段	IN_PORT MAC_SA/MAC_DA VLAN_ID/VLAN_PCP ETHER_TYPE ICMP TYPE/ICMP CODE ARP_OP/ARP_SPA/ARP_TPA IPSA/IPDA/IP_DSCP L3_PROTOCOL TCP_SRC_PORT/TCP_DST_PORT UDP_SRC_PORT/UDP_DST_PORT GRE TUNNEL ID MPLS_LABEL
精确匹配字段	N/A

流表行为	OUTPUT SET_FIELD(MAC_SA/MAC_DA/ VLAN_VID/VLAN_PCP/IPV4_DA/TCP_DST_PORT/UDP_D ST_PORT/MPLS_LABEL/MPLS_TC/TUNNEL_ID) PUSH_MPLS/POP_MPLS PUSH_VLAN/POP_VLAN DEC_IP_TTL SET_MPLS_TTL
Group	支持
MPLS	支持
GRE	支持
QinQ	支持
Meter	支持
Queue	支持
sFlow	支持

3 配置流表

3.1 概述

V350/V150 既支持通过传统 CLI 方式配置流表，也支持通过 OpenFlow 控制器来配置流表。

以下章节将通过示例来介绍如何通过 CLI 方式配置 OpenFlow 流表。

3.2 示例拓朴

在本节的配置示例中，将采用以下图例 1 所示的示例拓朴，交换机 1 分别负责接入主机 1、主机 2，交换机 2 分别负责接入主机 3、主机 4，两者之间通过端口 2（OpenFlow 端口号，对应交换机端口 eth-0-2，下同）互连，两个交换机通过一个集中的 OpenFlow 控制器来管理。

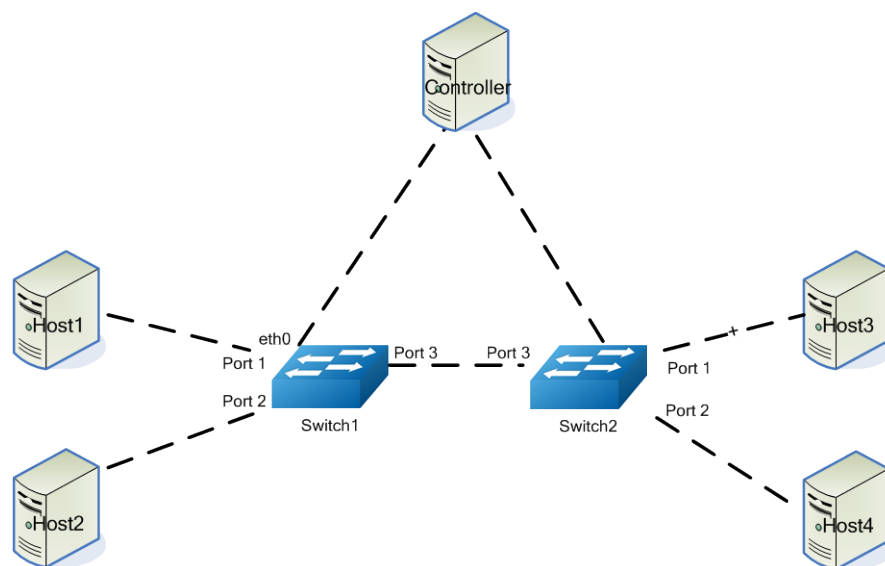


Figure 3-1 : OpenFlow 配置示例拓朴

3.3 配置转发到单个端口的流表

在交换机 1 上配置一条流表，该流表匹配从端口 1 上收到的源 mac 为 00:00:00:00:00:01 目的 mac 为 00:00:00:00:00:02 的 IPv4 报文，并将其转发到端口 2 上。

Switch 1

Switch1# ovs-ofctl add-flow br0 "in_port=1, dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02,dl_type=0x0800, actions=output:2" -O openflow13	将端口 1 上收到的指定报文转发到端口 2
--	-----------------------

配置验证

验证交换机 1 上的流表配置。

Switch1# ovs-ofctl dump-flows br0 -O openflow13

```
OFPPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=6.677s, table=0, n_packets=0, n_bytes=0,
  ip,in_port=1,dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:2
```

3.4 配置转发到逻辑端口 ALL 的流表

在交换机 1 上配置一条流表，该流表匹配从端口 1 上收到的带有 VLAN 10 并且 VLAN 优先级为 3 的 IPv4 报文，并将其转发到逻辑端口 ALL，从而将相应的报文广播到交换机所有端口上（不包括端口 1 以及配置了 OFPPC_NO_FWD 属性的端口）。

Switch 1

Switch1# ovs-ofctl add-flow br0 "in_port=1, dl_vlan=10, dl_vlan_pcp=3,dl_type=0x0800, actions=all" -O openflow13	将端口 1 上收到的指定报文转发到逻辑端口 ALL
---	---------------------------

配置验证

验证交换机 1 上的流表配置。

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPT_FLOW reply (OF1.3) (xid=0x4):  
  cookie=0x0, duration=2.039s, table=0, n_packets=0, n_bytes=0,  
  ip,in_port=1,dl_vlan=10,dl_vlan_pcp=3 actions=ALL
```

3.5 配置转发到控制器的流表

在交换机 1 上配置一条流表，该流表匹配从任意端口上收到的 IPv4 TOS 为 8 且源端口为 80，目的端口为 90 的 IPv4 TCP 报文，并将其转发到控制器。

Switch 1

Switch1# ovs-ofctl add-flow br0 "dl_type=0x0800,nw_tos=8,tcp,tp_src=80,tp_dst=90 actions=controller" -O openflow13	将任意端口上收到的指定 报文转发到控制器
--	-------------------------

配置验证

验证交换机 1 上的流表配置。

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPT_FLOW reply (OF1.3) (xid=0x4):  
  cookie=0x0, duration=4.567s, table=0, n_packets=0, n_bytes=0,  
  tcp,nw_tos=8,tp_src=80,tp_dst=90 actions=CONTROLLER:65535
```

3.6 配置丢弃报文的流表

在交换机 1 上配置一条流表，该流表匹配从端口 1 上收到 IPv4 报文，并将其丢弃。

Switch 1

Switch1# ovs-ofctl add-flow br0 "in_port=1, dl_type=0x0800, actions=drop" -O openflow13	丢弃从端口 1 上收到的 IPv4 报文
--	----------------------

配置验证

验证交换机 1 上的流表配置。

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=2.747s, table=0, n_packets=0, n_bytes=0, ip,in_port=1
  actions=drop
```

3.7 配置修改二层字段并转发到多个端口的流表

在交换机 1 上配置一条流表，该流表匹配从端口 1 上收到带有 VLAN 10，源 mac 为 00:00:00:00:00:01 目的 mac 为 00:00:00:00:00:02 的 IPv4 报文，修改其 VLAN 为 11，VLAN 优先级为 3，并修改其源 mac 为 00:00:00:00:00:03 目的 mac 为 00:00:00:00:00:04，将其转发到端口 2 和端口 3。

Switch 1

Switch1# ovs-ofctl add-flow br0 "in_port=1, dl_vlan=10, dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02,dl_type=0x0800, actions=set_field:11->vlan_vid,set_field:3->vlan_pcp,set_field:00:00:00:00:00:03->eth_src,set_field:00:00:00:00:00:04->eth_dst, output:2, output:3" -O openflow13	修改从端口 1 上收到的指定报文并将其转发到端口 2 和端口 3
---	----------------------------------

配置验证

验证交换机 1 上的流表配置。

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=2.79s, table=0, n_packets=0, n_bytes=0,
  ip,in_port=1,dl_vlan=10,dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02
  actions=set_field:11->vlan_vid,set_field:3->vlan_pcp,set_field:00:00:00:00:00:03->eth_src,
  set_field:00:00:00:00:00:04->eth_dst,output:2,output:3
```

3.8 配置修改三层字段并转发到多个端口的流表

在默认模式下，V350/V150 支持修改目的 IPv4 地址，此时流表必须匹配 IPv4 报文。

在交换机 1 上配置一条流表，该流表匹配从端口 1 上收到带有 VLAN 10，源 mac 为 00:00:00:00:00:01 目的 mac 为 00:00:00:00:00:02 的 IPv4 报文，修改其 VLAN 为 11，VLAN 优先级为 3，修改其源 mac 为 00:00:00:00:00:03 目的 mac 为 00:00:00:00:00:04，并修改其目的 IPv4 地址为 1.2.3.4，将其转发到端口 2 和端口 3。

Switch 1

Switch1# ovs-ofctl add-flow br0 "in_port=1, dl_vlan=10, dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02,dl_type=0x0800, actions=set_field:11->vlan_vid,set_field:00:00:00:00:00:03->eth_src,set_field:00:00:00:00:00:04->eth_dst,set_field:1.2.3.4->nw_dst, output:2, output:3" -O openflow13	修改从端口 1 上收到的指定报文并将其转发到端口 2 和端口 3
---	----------------------------------

配置验证

验证交换机 1 上的流表配置。

Switch1# ovs-ofctl dump-flows br0 -O openflow13

```
OFPST_FLOW reply (OF1.3) (xid=0x2):  cookie=0x0, duration=2.371s, table=0, n_packets=0, n_bytes=0, ip,in_port=1,dl_vlan=10,dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=set_field:11->vlan_vid,set_field:00:00:00:00:00:03->eth_src,set_field:00:00:00:00:00:04->eth_dst,set_field:1.2.3.4->ip_dst,output:2,output:3
```

3.9 配置修改四层字段并转发到多个端口的流表

在默认模式下，V350/V150 支持修改目的端口，此时流表必须匹配 IPv4 TCP 或 UDP 报文。

在交换机 1 上配置一条流表，该流表匹配从端口 1 上收到带有 VLAN 20，目的 IPv4 地址为 1.1.1.1 目的端口为 100 的 TCP 报文，并修改其目的 IPv4 地址为 1.2.3.4，将其转发到端口 2 和端口 3。

Switch 1

```
Switch1# ovs-ofctl add-flow br0 "in_port=1, dl_vlan=20,  
nw_dst=1.1.1.1,tp_dst=100,tcp,  
actions=set_field:1.2.3.4->nw_dst,  
set_field:1000->tp_dst,output:2,output:3" -O openflow13
```

修改从端口 1 上收到的指定报文并将其转发到端口 2 和端口 3

配置验证

验证交换机 1 上的流表配置。

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPST_FLOW reply (OF1.3) (xid=0x2):  
  cookie=0x0, duration=1.961s, table=0, n_packets=0, n_bytes=0,  
  tcp,in_port=1,dl_vlan=20,nw_dst=1.1.1.1,tp_dst=100  
  actions=set_field:1.2.3.4->ip_dst,set_field:1000->tcp_dst,output:2,output:3
```

3.10 配置带优先级和超时的流表

在交换机 1 上配置一条流表，该流表匹配从任意端口上收到的 IPv4 报文，优先级为 1000，将在空闲 60 秒后超时并被删除。

Switch 1

```
Switch1# ovs-ofctl add-flow br0 "priority=1000,  
idle_timeout=30, hard_timeout=60, ip,  
actions=output:2" -O openflow13
```

将任意端口上收到的 IPv4 报文转发到端口 2，并设置优先级和超时

配置验证

验证交换机 1 上的流表配置。

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPST_FLOW reply (OF1.3) (xid=0x4):  
  cookie=0x0, duration=2.263s, table=0, n_packets=0, n_bytes=0,  
  idle_timeout=30,hard_timeout=60,priority=1000,ip actions=output:2
```

3.11 修改流表的行为

在交换机 1 上配置一条流表，该流表匹配从端口 1 上收到的 IPv4 报文，将其转发到端口 2。

Switch 1

Switch1# ovs-ofctl add-flow br0 "in_port=1, ip, actions=output:2" -O openflow13	将端口 1 上收到的指定报文转发到端口 2
---	-----------------------

将该流表的行为修改为转发到端口 3。

Switch 1

Switch1# ovs-ofctl mod-flows br0 "in_port=1, ip, actions=output:3" -O openflow13	修改指定流表的行为
--	-----------

配置验证

验证交换机 1 上的流表配置。

```
Switch1# ovs-ofctl add-flow br0 "in_port=1, ip, actions=output:2" -O openflow13
```

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPT_FLOW reply (OF1.3) (xid=0x4):
 cookie=0x0, duration=1.927s, table=0, n_packets=0, n_bytes=0, ip,in_port=1
 actions=output:2
```

```
Switch1# ovs-ofctl mod-flows br0 "in_port=1, ip, actions=output:3" -O openflow13
```

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPT_FLOW reply (OF1.3) (xid=0x4):
 cookie=0x0, duration=6.439s, table=0, n_packets=0, n_bytes=0, ip,in_port=1
 actions=output:3
```

3.12 删除指定的流表

在交换机 1 上配置流表，两条流表匹配从端口 1 上收到的报文，另外两条流表匹配从端口 2 上收到的报文。

Switch 1

Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=80, actions=drop" -O openflow13	丢弃从端口 1 上收到的指定报文
Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=81, actions=drop" -O openflow13	丢弃从端口 1 上收到的指定报文
Switch1# ovs-ofctl add-flow br0 "in_port=2, tcp, tp_src=80, actions=drop" -O openflow13	丢弃从端口 2 上收到的指定报文
Switch1# ovs-ofctl add-flow br0 "in_port=2, tcp, tp_src=81, actions=drop" -O openflow13	丢弃从端口 2 上收到的指定报文

删除匹配端口 1 的流表。

Switch 1

Switch1# ovs-ofctl del-flows br0 "in_port=1" -O openflow13	删除匹配端口 1 的报文
--	--------------

配置验证

验证交换机 1 上的流表配置。

```
Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=80, actions=drop" -O openflow13
```

```
Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=81, actions=drop" -O openflow13
```

```
Switch1# ovs-ofctl add-flow br0 "in_port=2, tcp, tp_src=80, actions=drop" -O
openflow13
```

```
Switch1# ovs-ofctl add-flow br0 "in_port=2, tcp, tp_src=81, actions=drop" -O
openflow13
```

```
Switch1# ovs-ofctl dump-flows br0
OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=5.683s, table=0, n_packets=0, n_bytes=0, tcp,in_port=1,tp_src=80
  actions=drop
  cookie=0x0, duration=5.619s, table=0, n_packets=0, n_bytes=0, tcp,in_port=1,tp_src=81
  actions=drop
  cookie=0x0, duration=5.223s, table=0, n_packets=0, n_bytes=0, tcp,in_port=2,tp_src=81
  actions=drop
  cookie=0x0, duration=5.556s, table=0, n_packets=0, n_bytes=0, tcp,in_port=2,tp_src=80
  actions=drop
Switch1#
```

```
Switch1# ovs-ofctl del-flows br0 "in_port=1" -O openflow13
```

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=16.983s, table=0, n_packets=0, n_bytes=0,
  tcp,in_port=2,tp_src=81 actions=drop
  cookie=0x0, duration=17.316s, table=0, n_packets=0, n_bytes=0,
  tcp,in_port=2,tp_src=80 actions=drop
```

3.13 删除所有流表

在交换机 1 上配置流表，两条流表匹配从端口 1 上收到的报文，另外两条流表匹配从端口 2 上收到的报文。

Switch 1

Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=80, actions=drop" -O openflow13	丢弃从端口 1 上收到的指定报文
Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=81, actions=drop" -O openflow13	丢弃从端口 1 上收到的指定报文
Switch1# ovs-ofctl add-flow br0 "in_port=2, tcp, tp_src=80, actions=drop" -O openflow13	丢弃从端口 2 上收到的指定报文

```
Switch1# ovs-ofctl add-flow br0 "in_port=2, tcp, tp_src=81,  
actions=drop" -O openflow13
```

丢弃从端口 2 上收到的指
定报文

删除所有流表。

Switch 1

```
Switch1# ovs-ofctl del-flows br0 -O openflow13
```

删除所有流表

配置验证

验证交换机 1 上的流表配置。

```
Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=80, actions=drop" -O  
openflow13
```

```
Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=81, actions=drop" -O  
openflow13
```

```
Switch1# ovs-ofctl add-flow br0 "in_port=2, tcp, tp_src=80, actions=drop" -O  
openflow13
```

```
Switch1# ovs-ofctl add-flow br0 "in_port=2, tcp, tp_src=81, actions=drop" -O  
openflow13
```

```
Switch1# ovs-ofctl del-flows br0 -O openflow13
```

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPPST_FLOW reply (OF1.3) (xid=0x4):
```

3.14 查询单条流表统计

在交换机 1 配置两条匹配从端口 1 上收到的指定报文的流表。

Switch 1

Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=80, actions=drop" -O openflow13	丢弃从端口 1 上收到的指定报文
Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=81, actions=drop" -O openflow13	丢弃从端口 1 上收到的指定报文

查询流表统计。

Switch 1

Switch1# ovs-ofctl dump-flows br0 -O openflow13	查询流表统计
---	--------

配置验证

验证交换机 1 上的流表配置。

```
Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=80, actions=drop" -O
openflow13
```

```
Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=81, actions=drop" -O
openflow13
```

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```

OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=0.483s, table=0, n_packets=100, n_bytes=6400,
  tcp,in_port=1,tp_src=80 actions=drop
  cookie=0x0, duration=0.419s, table=0, n_packets=100, n_bytes=6400,
  tcp,in_port=1,tp_src=81 actions=drop
Switch1#

```

3.15 查询流表全局统计

在交换机 1 配置两条匹配从端口 1 上收到的指定报文的流表。

Switch 1

Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=80, actions=drop" -O openflow13	丢弃从端口 1 上收到的指定报文
Switch1# ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=81, actions=drop" -O openflow13	丢弃从端口 1 上收到的指定报文

查询流表全局统计。

Switch 1

Switch1# ovs-ofctl dump-tables br0 -O openflow13	查询流表全局统计
--	----------

配置验证

验证交换机 1 上的配置。

Switch1# **ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=80, actions=drop" -O openflow13**

Switch1# **ovs-ofctl add-flow br0 "in_port=1, tcp, tp_src=81, actions=drop" -O openflow13**

Switch1# **ovs-ofctl dump-tables br0 -O openflow13**

```

OFPST_TABLE reply (xid=0x1): 1 tables
  0: classifier: wild=0x3ffffff, max= 2560, active=2
      lookup=100, matched=100
Switch1#

```

4 配置 MPLS

4.1 概述

MPLS 是一种高度可扩展的，与协议无关的数据转发机制。传统 MPLS 网络中，通过往数据报文中插入一个 MPLS 标签，实现数据包的转发决策完全取决于该标签的内容，而不需要检查数据包本身。下面示例创建了一个跨越任意类型传输介质和协议的端到端的线路。

通过 OpenFlow 和 SDN，控制平面和 MPLS 的数据转发平面可以解耦。SDN 控制器之上的网络应用程序可以创建全局的 MPLS 服务，优化 MPLS 服务使其更加的灵活。

盛科 V350/V150 交换机软件扩展了 OpenFlow 1.3.1 规范以支持 MPLS 流表编程，它支持以下功能：

- IPv4 over MPLS 隧道
- 虚拟伪线路服务
- VPN 服务（注意：当前只支持将广播报文发送到各 PE 和 AC）

本节介绍如何通过 CLI 配置 IPv4 over MPLS 隧道，VPWS 及 VPLS 服务。

本节涉及的扩展 matching 字段均基于 Nicira。

4.2 参考资料

- OpenFlow 1.3.1 specification
- RFC 3031
- RFC 3032
- RFC 4447

- RFC 4762
- Open vSwitch

4.3 配置 IP over MPLS

V350/V150 支持通过 MPLS 封装 IPv4 报文，通过 MPLS 隧道将 IPv4 报文发送到远端设备。

以下配置示例中，会创建一条 LSP (Label Switching Path)。

通过 switch1, switch2, switch3 之间的 LSP, switch1 上 eth-0-1 收到的 IPv4 报文经过 MPLS 封装之后, 通过隧道发送到 switch3, 在 switch3 上报文经过解封装之后, 原始的 IPv4 报文会从 switch3 上的 eth-0-2 口发送出去。

以下是交换机上 MPLS 标签配置的简要说明:

- Switch1: 压入标签 16
- Switch2: 执行标签交换, 标签 16 与标签 17 交换
- Switch3: 弹出标签 17



Figure 4-1 : IP over MPLS 拓扑

Switch 1

Switch1# ovs-ofctl add-flow br0 "in_port=1,dl_type=0x800,actions=pop_l2, push_mpls:0x8847,set_field:16->mpls_label, push_l2,set_field:00:1e:08:00:02:01->eth_ dst,output:2" -O openflow13	弹出 eth-0-1 收到的 IPv4 报文的以太头， 压入 label 16，压入新的以太头，新以太头的 目的 mac 为 swith2 的 eth-0-1 的 mac 地 址，将修改后的报文从 eth-0-2 发送到 swith2
---	---

Switch 2

Switch2# ovs-ofctl add-flow br0 "dl_type=0x8847,mpls_label=16,actions=p op_l2,pop_mpls:0x800,push_mpls:0x8847, set_field:17->mpls_label,push_l2,set_field: 00:1e:08:00:03:01->eth_dst,output:2" -O openflow13	弹出 eth-0-1 收到的 MPLS 报文的以太头， 用 label 17 替换报文中的 16，压入新的以 太头，新以太头的目的 mac 地址为 swith3 的 eth-0-1 的 mac 地址，将修改后的报文 从 eth-0-2 发送到 swith3
--	--



说明

对于 L2VPN，“pop_mpls”的以太类型是未定义的，基于此，对所有 MPLS 流表，我们并不检查“pop_mpls”行为的以太类型，用户可以指定任意类型。

Switch 3

Switch3# ovs-ofctl add-flow br0 "dl_type=0x8847,mpls_label=17, actions=pop_l2,pop_mpls:0x800,push_l2,se t_field:00:00:00:00:04:01->eth_dst,output:2 " -O openflow13	弹出 eth-0-1 收到的 MPLS 报文的以太头， 弹出 lable 17，压入新的以太头，新以太 头的目的 mac 地址为下一跳设备 mac 地址 (未在 topo 图中列出)，该设备与 swith3 直连，从 eth-0-2 将报文发出
---	--



说明

Switch3 上 MPLS 报文解封装之后，V350/V150 芯片不能支持再次匹配 IP 流表，因此，用户需要显式的为已弹出 MPLS 标签的 flow 指定一个有效的出接口和下一跳的 MAC 地址。

验证配置

Step 1 验证 swith1 上的流表。

Switch1# **ovs-ofctl dump-flows br0 -O openflow13**

```
OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=4.085s, table=0, n_packets=0, n_bytes=0, ip,in_port=1
  actions=pop_l2,push_mpls:0x8847,set_field:
  16->mpls_label,push_l2,set_field:00:1e:08:00:02:01->eth_dst,output:2
```

Step 2 验证 swith2 上的流表。

Switch2# **ovs-ofctl dump-flows br0 -O openflow13**

```
OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=3.113s, table=0, n_packets=0, n_bytes=0,
  dl_type=0x8847,mpls_label=16
  actions=pop_l2,pop_mpls:0x0800,push_mpls:0x8847,set_field:17->mpls_label,push_l2,se
  t_field:00:1e:08:00:03:01->eth_dst,output:2
```

Step 3 验证 swith3 上的流表。

Switch3# **ovs-ofctl dump-flows br0 -O openflow13**

```
OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=4.273s, table=0, n_packets=0, n_bytes=0,
  dl_type=0x8847,mpls_label=17
  actions=pop_l2,pop_mpls:0x0800,push_l2,set_field:00:00:00:00:04:01->eth_dst,output:
  2
```

4.4 配置 VPWS

V350/V150 支持通过 MPLS 封装以太报文，并通过 MPLS 隧道将 IPv4 报文发送到远端设备，该特性可实现 VPWS 服务。

以下配置示例中，会创建一条 PW (Pseudo Wire)。

通过 switch1，switch2，switch3 之间创建的 PW，switch1 上 eth-0-1 收到的以太报文会通过 MPLS 封装起来，通过隧道发送到 swith3，在 swith3 上报文经过解封装之后，原始的以太报文会从 swith3 上的 eth-0-2 口发送出去。

以下是交换机上 MPLS 标签配置的简要说明：

- Switch1:压入 PW 标签 16，压入 tunnel 标签 300
- Switch2: tunnel 标签 300 与 tunnel 标签 400 交换
- Switch3:弹出 tunnel 标签 400，弹出 pw 标签 16



Figure 4-2 : VPWS 拓扑

Switch 1

<pre>Switch1# ovs-ofctl add-flow br0 "in_port=1, actions= push_mpls:0x8847,set_field:16->mpls_label, push_mpls:0x8847,set_field:300->mpls_label, push_l2,set_field:00:1e:08:00:02:01->eth_dst, output:2" -O openflow13</pre>	对收到的以太网报文，压入 PW 内层 label 16, 压入 tunnel label 300. 压入新的以太网头，新以太网头的目的 mac 为 switch2 上的 eth-0-1 的 mac 地址，将报文从 switch1 上的 eth-0-2 发送到 switch2
---	---

Switch 2

<pre>Switch2# ovs-ofctl add-flow br0 "dl_type=0x8847,mpls_label=300,actions= pop_l2,pop_mpls:0x8847,push_mpls:0x8847, set_field:400->mpls_label,push_l2,set_field:00:1e:08:00:03:01->eth_dst, output:2" -O openflow13</pre>	对 eth-0-1 收到的以太网报文，弹出以太网头，用 label 400 替换 label 300, 压入新以太网头，新以太网头的目的 mac 地址为 switch3 上的 eth-0-1 的 mac 地址，将报文从 switch 2 上的 eth-0-2 发送给 switch 3
---	--

Switch 3

<pre>Switch3# ovs-ofctl add-flow br0 "dl_type=0x8847,mpls_label=400, actions=pop_l2,pop_mpls:0x8847,output:0xffffffff7 " -O openflow13</pre>	弹出 eth-0-1 收到的 MPLS 报文的以太网头，弹出 label 400（见下面说明）
--	---

Switch3#ovs-ofctl add-flow br0 "dl_type=0x8847,mpls_label=16, actions=pop_l2,pop_mpls:0x800,output:2" -O openflow13	弹出外层隧道标签后，继续弹出内层 PW 标签，将报文从 eth-0-2 发出
--	--

说明

交换机 3 收到带有两层标签的 MPLS 报文之后，外层为隧道标签，会被弹出（flow 使用特定端口 0xffffffff7 弹出隧道标签），内层为 PW 标签，交换机会根据 PW 标签和流表中的相应配置来转发该 MPLS 报文。同时，流表中对 PW 标签的 pop_l2 编辑行为会被忽略，因为在对外层隧道标签操作中已经做了此动作。

验证配置

Step 1 验证 swith1 上的流表。

Switch1# **ovs-ofctl dump-flows br0 -O openflow13**

```

OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=1.592s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=push_mpls:0x8847,set_field:16->mpls_label,push_mpls:0x8847,set_field:300->m
  pls_label,push_l2,set_field:00:1e:08:00:02:01->eth_dst,output:2

```

Step 2 验证 swith2 上的流表。

Switch2# **ovs-ofctl dump-flows br0 -O openflow13**

```

OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=1.082s, table=0, n_packets=0, n_bytes=0,
  dl_type=0x8847,mpls_label=300
  actions=pop_l2,pop_mpls:0x8847,push_mpls:0x8847,set_field:400->mpls_label,push_l2,s
  et_field:00:1e:08:00:03:01->eth_dst,output:2

```

Step 3 验证 swith3 上的流表。

Switch3# **ovs-ofctl dump-flows br0 -O openflow13**

```

OFPST_FLOW reply (xid=0x4):
  cookie=0x0, duration=27.027s, table=0, n_packets=0, n_bytes=0,
  dl_type=0x8847,mpls_label=400 actions=pop_l2,pop_mpls:0x8847,PW_FWD
  cookie=0x0, duration=3.824s, table=0, n_packets=0, n_bytes=0,
  dl_type=0x8847,mpls_label=16 actions=pop_l2,pop_mpls:0x0800,output:2

```

4.5 配置 VPLS

OpenFlow 目前暂不支持定义 VSI (Virtual Switch Instance)，因此，V350/V150 上的 VPLS 是通过创建流表并向远端 PE 或 AC 广播以太网报文实现的。

以下配置示例中，交换机通过 PW 以全互联的方式连接，对每个交换机从 AC 上收到的以太网报文，将被封装在 MPLS 报文里向其他交换机广播。

以下是交换机上 MPLS 标签配置的简要说明：

- Switch1：对于发送给交换机 2 的报文，压入 PW 标签 16，隧道标签 300，对于发送给交换机 3 的报文，压入 PW 标签 16，隧道标签 400
- Switch2：弹出隧道标签 300，弹出 PW 标签 16
- Switch3：弹出隧道标签 400，弹出 PW 标签 16

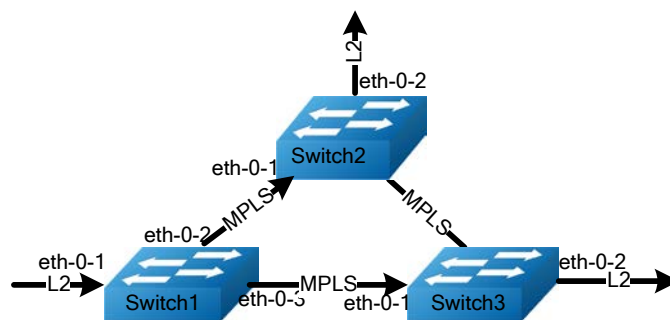


Figure 4-3 : VPLS 拓扑

Switch 1

<pre>Switch1# ovs-ofctl add-flow br0 "in_port=1, actions= push_mpls:0x8847,set_field:16->mpls_label, push_mpls:0x8847,set_field:300->mpls_label, push_l2,set_field:00:1e:08:00:02:01->eth_dst, output:2, push_mpls:0x8847,set_field:16->mpls_label, push_mpls:0x8847,set_field:400->mpls_label, push_l2,set_field:00:1e:08:00:03:01->eth_dst, output:3" -O openflow13</pre>	对 eth-0-1 收到的以太网报文，压入两层标签，内层是 PW 标签 16，外层是隧道标签 300，压入新的以太网头，以太网头中的目的 mac 为交换机 2 上的 eth-0-1 的 mac 地址，通过交换机 1 上的 eth-0-2 将报文发送给交换机 2，压入两层标签，内层是 PW 标签 16，外层隧道标签 400，压入新的以太网头，以太网头中的目的 mac 为交换机 3 上的 eth-0-1 的 mac 地址，通过交换机 1 上的 eth-0-3 将报文发送给交换机 3
---	--

Switch 2

<pre>Switch2# ovs-ofctl add-flow br0 " dl_type=0x8847,mpls_label=300, actions=pop_l2,pop_mpls:0x8847, output:0xffffffff7" -O openflow13</pre>	弹出 eth-0-1 收到的 MPLS 报文的以太网头，弹出外层隧道标签 300
<pre>Switch2# ovs-ofctl add-flow br0 "dl_type=0x8847,mpls_label=16, actions=pop_l2,pop_mpls:0x800,output:2" -O openflow13</pre>	弹出外层隧道标签之后，再弹出内层 PW 标签，将报文从 eth-0-2 发出去

Switch 3

<pre>Switch3# ovs-ofctl add-flow br0 " dl_type=0x8847,mpls_label=400, actions=pop_l2,pop_mpls:0x8847, output:0xffffffff7" -O openflow13</pre>	弹出 eth-0-1 收到的 MPLS 报文的以太网头，弹出外层隧道标签 400
---	---

Switch3# ovs-ofctl add-flow br0 "dl_type=0x8847,mpls_label=16, actions=pop_l2,pop_mpls:0x800,output:2" -O openflow13	弹出外层隧道标签之后，再弹出内层 PW 标签，将报文从 eth-0-2 发出去
---	---

验证配置

Step 1 验证交换机 1 上配置的流表。

Switch1# ovs-ofctl dump-flows br0 -O openflow13

```

OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=2.69s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=push_mpls:0x8847,set_field:16->mpls_label,push_mpls:0x8847,set_field:300->m
  pls_label,push_l2,set_field:00:1e:08:00:02:01->eth_dst,output:2,push_mpls:0x8847,se
  t_field:16->mpls_label,push_mpls:0x8847,set_field:400->mpls_label,push_l2,set_field
  :00:1e:08:00:03:01->eth_dst,output:3

```

Step 2 验证交换机 2 上配置的流表。

Switch2# ovs-ofctl dump-flows br0 -O openflow13

```

OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=11.7s, table=0, n_packets=0, n_bytes=0,
  dl_type=0x8847,mpls_label=300 actions=pop_l2,pop_mpls:0x8847,PW_FWD
  cookie=0x0, duration=3.233s, table=0, n_packets=0, n_bytes=0,
  dl_type=0x8847,mpls_label=16 actions=pop_l2,pop_mpls:0x0800,output:2

```

Step 3 验证交换机 3 上配置的流表。

Switch3# ovs-ofctl dump-flows br0 -O openflow13

```

OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=12.732s, table=0, n_packets=0, n_bytes=0,
  dl_type=0x8847,mpls_label=400 actions=pop_l2,pop_mpls:0x8847,PW_FWD
  cookie=0x0, duration=2.877s, table=0, n_packets=0, n_bytes=0,
  dl_type=0x8847,mpls_label=16 actions=pop_l2,pop_mpls:0x0800,output:2

```

4.6 OpenFlow Spec 1.3 修改

以下列出了 OpenFlow Spec 1.3 上 MPLS 实现上的修改。

Modification Description	Modification in C source code
定义一个特殊的逻辑端口来抽象表示所有 MPLS 隧道的出端口，因为 L2VPN 报文是通过 MPLS 隧道转发的，V350/V150 在弹出外层隧道标签后，要进行下一跳标签查找(即对应的 VPWS/VPLS 等的 PW 标签)，通过下一跳标签确定从哪个端口发出去，因此 MPLS 隧道的出方向 flow 上配置的出端口是没有意义的。	在枚举结构 ofp_port_no 中，增加新的逻辑接口： OFPP_PW_FWD = 0xffffffff7

4.7 基于 Nicira 的扩展流表行为

以下列出了 V350/V150 上基于 Nicira 的扩展流表行为。

扩展 action 类型	Action 定义	ovs-ofctl 命令行语法
PUSH_L2 = 25	<pre>struct nx_action_push_l2 { ovs_be16 type; /* OFPAT_VENDOR. */ ovs_be16 len; /* Length is 8. */ ovs_be32 vendor; /* NX_VENDOR_ID. */ ovs_be16 subtype; /* PUSH_L2 */ uint8_t pad[6]; };</pre>	push_l2
POP_L2 = 26	<pre>struct nx_action_pop_l2 { ovs_be16 type; /* OFPAT_VENDOR. */ ovs_be16 len; /* Length is 8. */ ovs_be32 vendor; /* NX_VENDOR_ID. */ ovs_be16 subtype; /* POP_L2 */ uint8_t pad[6]; };</pre>	pop_l2

4.8 MPLS 支持的 action

以下列出了 MPLS 支持的所有 action。

相关语法说明：

- []: 表示可选项。
- (): 用于 group 相关的流表行为。
- |: []中的“|”表示单选。

服务类型	Action 列表	出接口	注释
IP over MPLS ingress	NXAST_POP_L2 PUSH_MPLS [SET_FIELD->MPLS_TC SET_MPLS_TTL] SET_FIELD->MPLS_LABEL PUSH_L2 SET_FIELD->DL_DST [SET_FIELD->VLAN_ID] OFPAT_OUTPUT	物理口	action 中的 TC 和 TTL 是可选的
IP over MPLS transit	NXAST_POP_L2 POP_MPLS PUSH_MPLS [SET_FIELD->MPLS_TC SET_MPLS_TTL] SET_FIELD->MPLS_LABEL PUSH_L2 SET_FIELD->DL_DST [SET_FIELD->VLAN_ID] OFPAT_OUTPUT	物理口	action 中的 TC 和 TTL 是可选的
IP over MPLS egress	NXAST_POP_L2 POP_MPLS NXAST_PUSH_L2 SET_FIELD->DL_DST [SET_FIELD->VLAN_ID] OFPAT_OUTPUT	物理口	-
MPLS tunnel egress	NXAST_POP_L2 POP_MPLS OFPAT_OUTPUT	OFPP_PW_FWD 类型接口	该类型的流只用于 L2VPN 中对隧道报文解封封装

服务类型	Action 列表	出接口	注释
VPWS ingress	[POP_VLAN (PUSH_VLAN SET_FIELD->VLAN_ID) SET_FIELD->VLAN_ID] PUSH_MPLS [SET_FIELD->MPLS_TC SET_MPLS_TTL] SET_FIELD->MPLS_LABEL PUSH_MPLS [SET_FIELD->MPLS_TC SET_MPLS_TTL] SET_FIELD->MPLS_LABEL NXAST_PUSH_L2 SET_FIELD->DL_DST [SET_FIELD->VLAN_ID] OFPAT_OUTPUT	物理口	1. action 中的 TC 和 TTL 是可选的 2. [POP_VLAN (PUSH_VLAN SET_FIELD->VLAN_ID) SET_FIELD->VLAN_ID] 以上多个选项不能共存，只能选择一个
VPWS egress	NXAST_POP_L2 POP_MPLS [POP_VLAN (PUSH_VLAN SET_FIELD->VLAN_ID) SET_FIELD->VLAN_ID] OFPAT_OUTPUT	物理口	1. [POP_VLAN (PUSH_VLAN SET_FIELD->VLAN_ID) SET_FIELD->VLAN_ID] 以上多个选项不能共存，只能选择一个
VPLS ingress	Multiple VPWS ingress action list combined	物理口	-
VPLS egress	Multiple VPWS egress action list combined	物理口	-

5 配置 GRE 隧道

5.1 概述

V350/V150 支持 GRE 隧道技术，用户可以通过配置 GRE 隧道来创建 Overlay 网络，提升网络的扩展性。

用户可以通过 CLI 静态配置，或利用 OVSDB 协议远程配置 GRE 隧道，这些隧道需要以“gre1”、“gre2”这样的形式命名（不同流表模式下支持的隧道数量不一），隧道的 OpenFlow 逻辑端口号以 201 开始，该逻辑端口可用于流表的入端口匹配或出端口转发。

如果流表转发到 GRE 隧道对应的逻辑端口，则该报文将被封装在 GRE 报文里进行转发，GRE 报文的外层 IPv4 头、以太头将按照创建 GRE 隧道时的配置进行设置。

V350/V150 通过扩展的 GRE_DECAP 逻辑端口（OpenFlow 1.0 模式时为 0xfff6，OpenFlow 1.3 模式时为 0xfffff6）来实现 GRE 报文的解封装，用户可通过配置流表将匹配的 GRE 报文转发到该逻辑端口来实现按需解封装。

5.2 配置 GRE 隧道

在下面的配置示例中，在 交换机 1 与交换机 2 间创建一条 GRE 隧道。在交换机 1 上，隧道的本端地址配置为 1.1.1.1，对端地址配置为 2.2.2.2，该隧道绑定在端口 2 上，目的 mac 地址为 00:00:00:00:00:02；在交换机 2 上，隧道的本端地址配置为 2.2.2.2，对端地址配置为 1.1.1.1，该隧道绑定在端口 2 上，目的 mac 地址为 00:00:00:00:00:01。



由于 V350/V150 是纯 OpenFlow 交换机，因此隧道的目的 mac 用户可以根据需求灵活配置，对端可以针对该 mac 按需进行匹配处理。

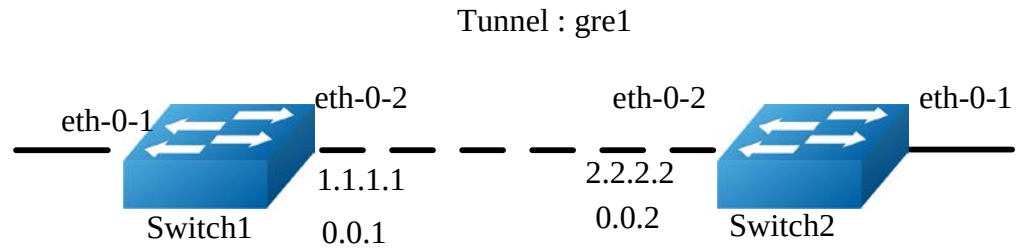


Figure 5-1 : GRE Tunnel Topology

Switch 1

Switch1# ovs-vsctl add-port br0 gre1 -- set interface gre1 type=gre options:remote_ip=2.2.2.2 options:local_ip=1.1.1.1 options:bind_port=eth-0-2 options:nexthop_mac=0.0.2	创建 GRE 隧道 1
Switch1# ovs-ofctl add-flow br0 in_port=1,actions=output:201 -O openflow13	将端口 1 上收到的所有报文通过 GRE 隧道 1 封装转发出去
Switch1# ovs-ofctl add-flow br0 priority=32769,in_port=2,gre,actions=gre_decap -O openflow13	添加一条流表，将端口 2 上收到的 GRE 报文解封装
Switch1# ovs-ofctl add-flow br0 in_port=201,actions=output:1 -O openflow13	将从 GRE 隧道 1 上解封装后的所有报文转发到端口 1

Switch 2

Switch2# ovs-vsctl add-port br0 gre1 -- set interface gre1 type=gre options:remote_ip=1.1.1.1 options:local_ip=2.2.2.2 options:bind_port=eth-0-2 options:nexthop_mac=0.0.1	创建 GRE 隧道 1
Switch2# ovs-ofctl add-flow br0 in_port=1,actions=output:201 -O openflow13	将端口 1 上收到的所有报文通过 GRE 隧道 1 封装转发出去

Switch2# ovs-ofctl add-flow br0 priority=32769,in_port=2,gre,actions=gre_decap -O openflow13	添加一条流表，将端口 2 上收到的 GRE 报文解封装
Switch2# ovs-ofctl add-flow br0 in_port=201,actions=output:1 -O openflow13	将从 GRE 隧道 1 上解封装后的所有报文转发到端口 1

说明

- 1、配置 GRE 隧道时，除上面列出的属性外，用户可以指定一个 `vlan_id` 的属性（如 `options:vlan_id:10`），以便设置 GRE 报文二层头所携带的 VLAN。
- 2、匹配 GRE 逻辑端口进行解封装的流表，如果流表中未指定匹配 `tunnel id` 字段，则只能匹配不带 `tunnel id` 的 GRE 报文。

配置验证

Step 1 验证交换机 1 上的 GRE 隧道配置。

Switch1# **show interface tunnel brief**

The Maximum of tunnel ports is 200, currently 1 tunnel port is valid.

index	type	name	port	bind_port	local_ip	remote_ip	remote_mac	vlan_id	link
0	GRE	gre1	201	eth-0-2	1.1.1.1	2.2.2.2	00:00:00:00:00:02	None	UP

Switch1# **ovs-ofctl dump-flows br0 -O openflow13**

```
OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=35.666s, table=0, n_packets=0, n_bytes=0,
  priority=32769,gre,in_port=2 actions=GRE_DECAP
  cookie=0x0, duration=25.7s, table=0, n_packets=0, n_bytes=0, in_port=201
  actions=output:1
  cookie=0x0, duration=51.615s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=output:201
```

Step 2 验证交换机 2 上的 GRE 隧道配置。

Switch2# **show interface tunnel brief**

The Maximum of tunnel ports is 200, currently 1 tunnel port is valid.

index	type	name	port	bind_port	local_ip	remote_ip	remote_mac	vlan_id	link
0	GRE	gre1	201	eth-0-2	2.2.2.1	1.1.1.1	00:00:00:00:00:01	None	UP

Switch2# **ovs-ofctl dump-flows br0 -O openflow13**

```
OFPT_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=13.235s, table=0, n_packets=0, n_bytes=0, in_port=201
  actions=output:1
  cookie=0x0, duration=1.218s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=output:201
  cookie=0x0, duration=55.811s, table=0, n_packets=0, n_bytes=0,
  priority=32769,gre,in_port=2 actions=GRE_DECAP
```

5.3 配置 GRE 隧道 id

流表在将报文进行 GRE 封装前，可以指定一个隧道 id，该隧道 id 用来设置 GRE 头中的 GRE key 字段，一条 GRE 隧道上可以承载多个隧道 id。

流表可以在匹配隧道之外，还可以匹配该隧道上的隧道 id。

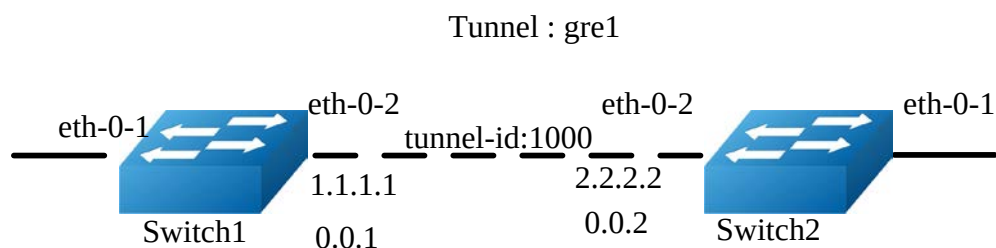


Figure 5-2 : GRE Tunnel Topology

Switch 1

Switch1# ovs-vsctl add-port br0 gre1 -- set interface gre1 type=gre options:remote_ip=2.2.2.2 options:local_ip=1.1.1.1 options:bind_port=eth-0-2 options:nexthop_mac=0.0.2	创建 GRE 隧道 1
Switch1# ovs-ofctl add-flow br0 "in_port=1,actions=set_field:1000->tun_id,output:201" -O openflow13	将端口 1 上收到的所有 报文通过 GRE 隧道 1 封装转发出去，同时设 置隧道 id 为 1000
Switch1# ovs-ofctl add-flow br0 priority=32769,in_port=2,gre,actions=gre_decap -O openflow13	添加一条流表，将端口 2 上收到的 GRE 报文解 封装

Switch1# ovs-ofctl add-flow br0 "in_port=201,tun_id=1000,actions=output:1" -O openflow13	将 GRE 隧道 1 收到的 上带有隧道 id 1000 的 GRE 报文解封装后转 发到端口 1
---	--

Switch 2

Switch2# ovs-vsctl add-port br0 gre1 -- set interface gre1 type=gre options:remote_ip=1.1.1.1 options:local_ip=2.2.2.2 options:bind_port=eth-0-2 options:nexthop_mac=0.0.1	创建 GRE 隧道 1
Switch2# ovs-ofctl add-flow br0 "in_port=1,actions=set_field:1000->tun_id,output:201" -O openflow13	将端口 1 上收到的所有 报文通过 GRE 隧道 1 封装转发出去，同时设 置隧道 id 为 1000
Switch2# ovs-ofctl add-flow br0 priority=32769,in_port=2,gre,actions=gre_decap -O openflow13	添加一条流表，将端口 2 上收到的 GRE 报文解 封装
Switch2# ovs-ofctl add-flow br0 "in_port=201,tun_id=1000,actions=output:1" -O openflow13	将 GRE 隧道 1 收到的 上带有隧道 id 1000 的 GRE 报文解封装后转 发到端口 1

配置验证

Step 1 验证交换机 1 上的 GRE 隧道配置。

Switch1# **show interface tunnel brief**

The Maximum number of tunnel ports is 200, currently 1 tunnel port is valid.

index	type	name	port	bind_port	local_ip	remote_ip	remote_mac	vlan_id	link
0	GRE	gre1	201	eth-0-2	1.1.1.1	2.2.2.2	00:00:00:00:00:02	None	UP

Switch1# **ovs-ofctl dump-flows br0 -O openflow13**

```
OFPST_FLOW reply (xid=0x4):
  cookie=0x0, duration=33.991s, table=0, n_packets=0, n_bytes=0,
  tun_id=0x3e8,in_port=201 actions=set_field:0x3e8->tun_id,output:1
  cookie=0x0, duration=38.059s, table=0, n_packets=0, n_bytes=0,
  priority=32769,gre,in_port=2 actions=GRE_DECAP
  cookie=0x0, duration=42.819s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=set_field:0x3e8->tun_id,output:201
```

Step 2 验证交换机 2 上的 GRE 隧道配置。

Switch2# **show interface tunnel brief**

The Maximum number of tunnel ports is 200, currently 1 tunnel port is valid.

index	type	name	port	bind_port	local_ip	remote_ip	remote_mac	vlan_id	link
0	GRE	gre1	201	eth-0-2	2.2.2.1	1.1.1.1	00:00:00:00:00:01	None	UP

Switch2# **ovs-ofctl dump-flows br0 -O openflow13**

```
OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=10.453s, table=0, n_packets=0, n_bytes=0,
  tun_id=0x3e8,in_port=201 actions=output:1
  cookie=0x0, duration=15.04s, table=0, n_packets=0, n_bytes=0,
  priority=32769,gre,in_port=2 actions=GRE_DECAP
  cookie=0x0, duration=19.445s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=set_field:0x3e8->tun_id,output:201
```

5.4 Update GRE tunnel nexthop MAC

某些场景中，比如 VRRP，用户可能会修改 GRE 隧道的下一跳 MAC 地址，当 flow 的出接口是 GRE 隧道时，会使用新的 MAC 地址。

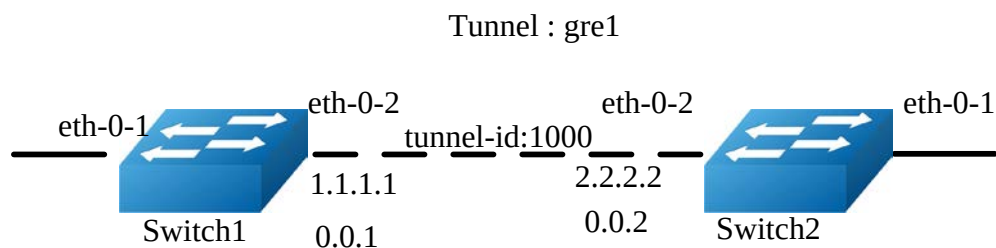


Figure 5-3 : GRE 隧道拓扑

Switch 1

Switch1# ovs-vsctl add-port br0 gre1 -- set interface gre1 type=gre options:remote_ip=2.2.2.2 options:local_ip=1.1.1.1 options:bind_port=eth-0-2 options:nexthop_mac=0.0.2	创建 GRE 隧道 gre1
Switch1# ovs-ofctl add-flow br0 "in_port=1,actions=set_field:1000->tun_id,output:201" -O openflow13	eth-0-1 收到的报文重定向到 tunnel 隧道并进行报文封装
Switch1# ovs-ofctl add-flow br0 priority=32769,in_port=2,gre,actions=gre_decap -O openflow13	添加一条 flow，解封装从 eth-0-2 收到的 gre 报文
Switch1# ovs-ofctl add-flow br0 "in_port=201,tun_id=1000,actions=output:1" -O openflow13	tunnel 端口收到的报文重定向到 eth-0-1 并进行报文封装

Switch 2

Switch2# ovs-vsctl add-port br0 gre1 -- set interface gre1 type=gre options:remote_ip=1.1.1.1 options:local_ip=2.2.2.2 options:bind_port=eth-0-2 options:nexthop_mac=0.0.1	创建 GRE 隧道 gre1
Switch2# ovs-ofctl add-flow br0 "in_port=1,actions=set_field:1000->tun_id,output:201" -O openflow13	eth-0-1 收到的报文重定向到 tunnel 隧道并进行报文封装
Switch2# ovs-ofctl add-flow br0 priority=32769,in_port=2,gre,actions=gre_decap -O openflow13	添加一条 flow，解封装从 eth-0-2 收到的 gre 报文
Switch2# ovs-ofctl add-flow br0 "in_port=201,tun_id=1000,actions=output:1" -O openflow13	tunnel 端口收到的报文重定向到 eth-0-1 并进行报文封装

Validation commands

Step 1 验证交换机 1 上的 GRE 隧道配置。

Switch1# **show interface tunnel brief**

The Maximum number of tunnel ports is 200, currently 1 tunnel port is valid.

index	type	name	port	bind_port	local_ip	remote_ip	remote_mac	vlan_id	link
0	GRE	gre1	201	eth-0-2	1.1.1.1	2.2.2.2	00:00:00:00:00:02	None	UP

Switch1# **ovs-ofctl dump-flows br0 -O openflow13**

```
OFPST_FLOW reply (xid=0x4):
  cookie=0x0, duration=33.991s, table=0, n_packets=0, n_bytes=0,
  tun_id=0x3e8,in_port=201 actions=set_field:0x3e8->tun_id,output:1
  cookie=0x0, duration=38.059s, table=0, n_packets=0, n_bytes=0,
  priority=32769,gre,in_port=2 actions=GRE_DECAP
  cookie=0x0, duration=42.819s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=set_field:0x3e8->tun_id,output:201
```

Step 2 验证交换机 2 上的 GRE 隧道配置。

Switch2# **show interface tunnel brief**

The Maximum number of tunnel ports is 200, currently 1 tunnel port is valid.

index	type	name	port	bind_port	local_ip	remote_ip	remote_mac	vlan_id	link
0	GRE	gre1	201	eth-0-2	2.2.2.1	1.1.1.1	00:00:00:00:00:01	None	UP

Switch2# **ovs-ofctl dump-flows br0 -O openflow13**

```
OFPST_FLOW reply (OF1.3) (xid=0x4):
  cookie=0x0, duration=10.453s, table=0, n_packets=0, n_bytes=0,
  tun_id=0x3e8,in_port=201 actions=output:1
  cookie=0x0, duration=15.04s, table=0, n_packets=0, n_bytes=0,
  priority=32769,gre,in_port=2 actions=GRE_DECAP
```

Step 3 修改交换机 1 上的 GRE 隧道的下一跳 mac。

Switch1# **show interface tunnel brief**

The Maximum number of tunnel ports is 200, currently 1 tunnel port is valid.

index	type	name	port	bind_port	local_ip	remote_ip	remote_mac	vlan_id	link
0	GRE	gre1	201	eth-0-2	1.1.1.1	2.2.2.2	00:00:00:00:00:02	None	UP

Switch1# **ovs-vsctl set interface gre1 options:nexthop_mac 0.0.3**

Switch1# **show interface tunnel brief**

The Maximum number of tunnel ports is 200, currently 1 tunnel port is valid.

index	type	name	port	bind_port	local_ip	remote_ip	remote_mac	vlan_id	link
0	GRE	gre1	201	eth-0-2	1.1.1.1	2.2.2.2	00:00:00:00:00:03	None	UP

Switch1# **ovs-ofctl dump-flows br0 -O openflow13**

```
OFPST _FLOW reply (xid=0x4):
  cookie=0x0, duration=33.991s, table=0, n_packets=0, n_bytes=0,
  tun_id=0x3e8,in_port=201 actions=set_field:0x3e8->tun_id,output:1
  cookie=0x0, duration=38.059s, table=0, n_packets=0, n_bytes=0,
  priority=32769,gre,in_port=2 actions=GRE_DECAP
  cookie=0x0, duration=42.819s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=set_field:0x3e8->tun_id,output:201
```

6 配置控制器

6.1 概述

本章将介绍三款流行的 OpenFlow 控制器的安装及使用，这三款控制器分别是 OpenDaylight、ryu 以及 floodlight。

6.2 示例拓扑

以下的图例介绍了两种典型的 OpenFlow 网络拓扑。

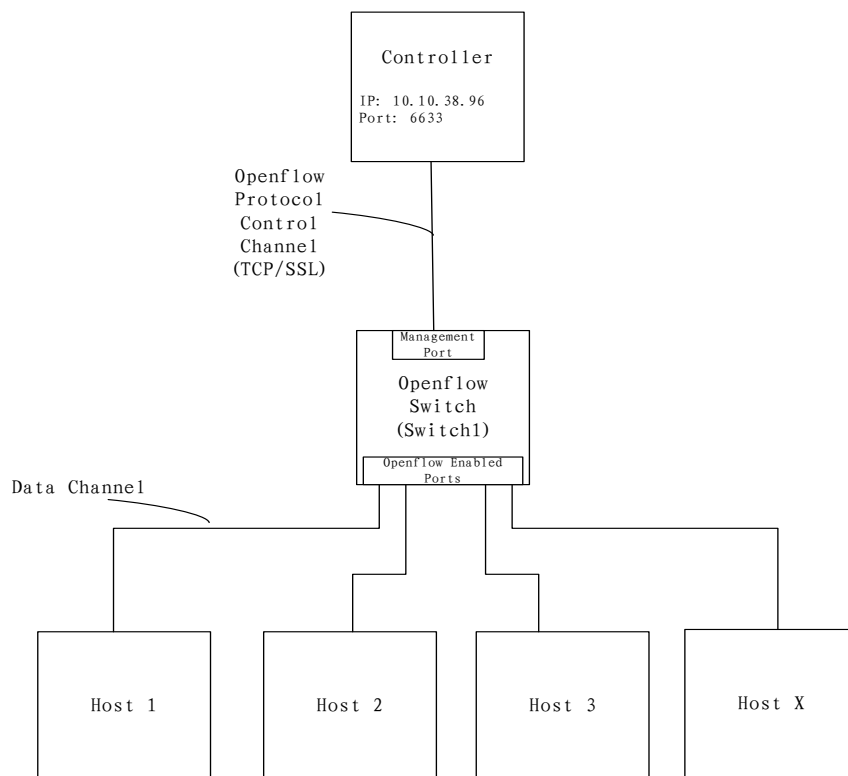


Figure 6-1：采用一个控制器 OpenFlow 网络

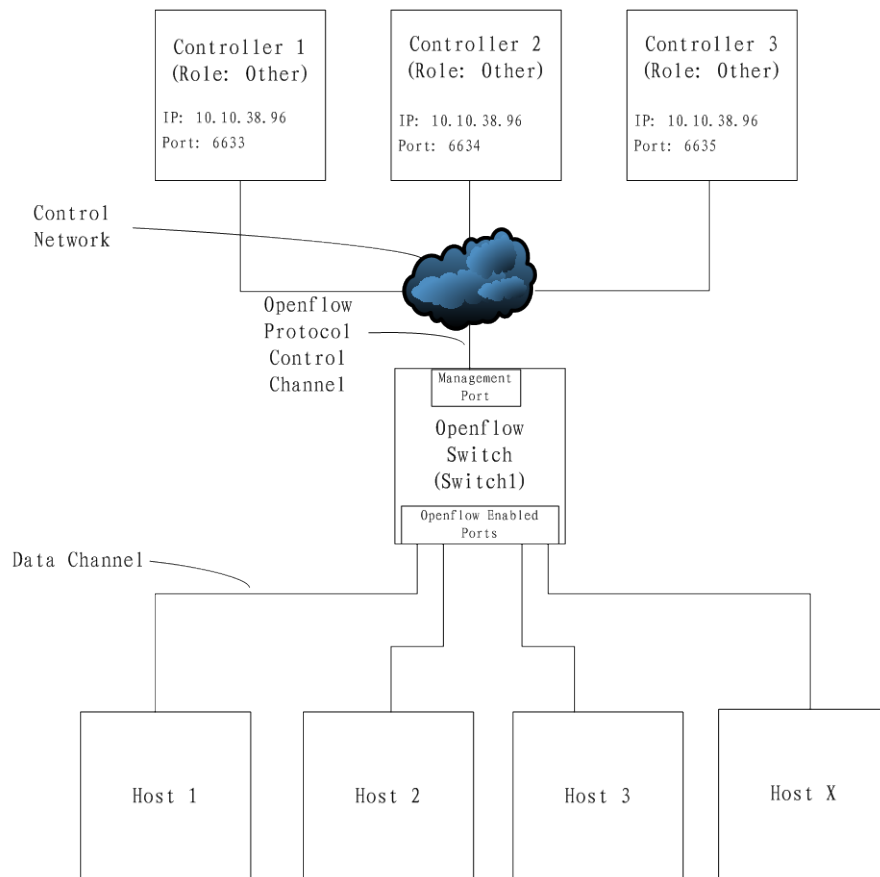


Figure 6-2：采用三个控制器 OpenFlow 网络

6.3 控制器配置

以下分别介绍前述三款控制器的使用。

6.3.1 OpenDaylight

OpenDaylight 是一款社区驱动、业界支持的开源控制器，目前受到了广泛的关注，其愿景是通过孕育不断的应用和创新来加速 SDN 的落地。

OpenDaylight 由 Linux 基金会负责运营，采用 Eclipse Public License 发布，使用 Java 编写。

下载控制器

用户可通过如下地址下载 OpenDaylight 控制器：

<http://www.opendaylight.org/software/downloads>.

安装控制器

Step 1 解压软件包

```
root@dslab-ubuntu10:~/opendaylight # unzip distributions-base-0.1.1-osgipackage.zip
```

Step 2 进入目录

```
root@dslab-ubuntu10:~/ opendaylight # cd opendaylight
```

操作控制器

以 OpenFlow 1.3 模式运行控制器（默认控制器端口为 6653）。

```
root@dslab-ubuntu10:~/opendaylight/opendaylight # run.sh -Of13
```

OpenDaylight 为不同服务提供了丰富的北向 REST API 接口，关于接口的详细信息，请参考如下 OpenDaylight 官方网站：

https://wiki.opendaylight.org/view/OpenDaylight_Controller:REST_Reference_and_Authentication

以下是使用 OpenDaylight 的 switchmanager 以及 flow programmer 的 REST API 来增删流表的示例。

增删流表

Step 1 通过 switchmanager REST API 来取得交换机的 node id。

```
curl -u admin:admin http://localhost:8080/controller/nb/v2/switchmanager/default/nodes
```

```
{ "nodeProperties": [ { "properties": { "tables": { "value": 1 }, "timeStamp": { "value": 1402906745493, "name": "connectedSince" }, "buffers": { "value": 16384 }, "capabilities": { "value": 7 }, "macAddress": { "value": "00:1e:08:00:06:17" } }, "node": { "id": "openflow:128983238167", "type": "MD_SAL" } } ] }
```

Step 2 添加一条匹配 HTTP 报文并丢弃报文的流表。

```
curl -u admin:admin -H 'Content-type: application/json' -X PUT -d '{ "installInHw": "true", "name": "flow1", "node": { "id": "openflow:128983238167", "type": "MD_SAL", "etherType": "0x800", "tpDst": "80", "priority": "500", "流表行为": ["DROP"] } }'  
http://localhost:8080/controller/nb/v2/flowprogrammer/default/node/MD_SAL/openflow:128983238167/staticFlow/flow1
```

```
Success
```

Step 3 删除上步中新增加的流表。

```
curl -u admin:admin -X DELETE
http://localhost:8080/controller/nb/v2/flowprogrammer/default/node/MD_SAL/openflow:128
983238167/staticFlow/flow1
```

```
无输出表示删除成功
```

6.3.2 ryu

Ryu 一款基于组件机制的 SDN 框架。

Ryu 提供了具有良好 API 定义的丰富组件,以便于开发者开发新的网络管理和控制应用。

Ryu 支持包括 OpenFlow、Netconf、OF-Config 等多种协议来管理交换机。

Ryu 采用 Apache 2.0 协议发布,使用 Python 编写。

下载控制器

用户可通过如下网址下载 ryu 控制器:

<http://osrg.github.io/ryu>.

安装控制器

Step 1 解压软件包

```
root@dslab-ubuntu10:~/ryu-3.10# unzip ryu-3.10.tar.gz
```

Step 2 进入目录

```
root@dslab-ubuntu10:~/ryu-3.10 # cd ryu-3.10
```

Step 3 安装 ryu

```
root@dslab-ubuntu10:~/ryu-3.10/ryu-3.10 # python setup.py install
```

操作控制器

使用该控制器加载自带的 ofctrl_rest 应用 (默认控制器端口为 6633)。

```
root@dslab-ubuntu10:~/ryu-3.10/ryu-3.10# ryu-manager ryu/app/ofctrl_rest.py
```

Ryu 自带了丰富的应用，关于应用的更多信息，请阅读 ryu 的源码以及下面网址中的相应文档：

<http://ryu.readthedocs.org/en/latest/index.html>

以下是使用 ofctl_rest 应用的 REST API 来增删流表的示例。

增删流表

Step 1 通过 REST API 获取交换机 dpid

```
curl http://localhost:8080/stats/switches
```

```
[128983238167]
```

Step 2 添加流表匹配 HTTP 报文并将其送往控制器

```
curl -X POST -d '{"dpid": "128983238167", "match": {"eth_type": 0x800, "ip_proto": 6, "tp_dst": 80}, "priority": 500, "流表行为": [{"type": "OUTPUT", "port": 0xffffffff}]}'  
http://localhost:8080/stats/flowentry/add
```

```
无输出表示添加成功
```

Step 3 删除上步中添加的流表

```
curl -X POST -d '{"dpid": "128983238167", "match": {"eth_type": 0x800, "ip_proto": 6, "tp_dst": 80}, "priority": 500}' http://localhost:8080/stats/flowentry/delete
```

```
无输出表示删除成功
```

6.3.3 floodlight

Floodlight 是一款企业级的控制器，它由一部分来自于 BigSwitch 公司的工程师支持。

下载控制器

用户可通过如下网址下载 floodlight 控制器：

<http://floodlight.openflowhub.org/download>.

安装控制器

Step 1 解压软件包

```
root@dslab-ubuntu10:~/floodlight_0.82# unzip flooglight0.82.zip
```

Step 2 进入目录

```
root@dslab-ubuntu10:~/floodlight_0.82# cd flooglight0.82
```

操作控制器

运行控制器（默认控制器端口为 6633）。

```
root@dslab-ubuntu10:~/floodlight_0.82# java -jar floodlight-0.82.jar
```

关于 Floodlight API 的细节，请参考下表。

API 汇总

URI	描述	参数
/wm/staticflowentrypusher/json	增删流表	HTTP POST 数据(增加流表), HTTP DELETE (删除流表)
/wm/staticflowentrypusher/list/<switch>/json	列出指定交换机或所有交换机上的流表	switch: 有效的 dpid (XX:XX:XX:XX:XX:XX:XX) or "all"
/wm/staticflowentrypusher/clear/<switch>/json	清除指定交换机或所有交换机上的流表	switch: 有效的 dpid (XX:XX:XX:XX:XX:XX:XX) or "all"

增删流表

Step 1 Floodlight 的 Static Flow Pusher 提供了 REST API 访问机制，使用该 REST API 来下发一条流表

```
curl -d '{"switch": "00:00:00:00:00:00:01", "name": "flow-mod-1", "cookie": "0",  
"priority": "32768", "ingress-port": "1", "active": "true", "流表行为": "output=2"}'  
http://<controller_ip>:8080/wm/staticflowentrypusher/json
```

关于流表的参数，请参考 Floodlight 的官方文档。

Step 2 删除上步添加的流表

```
curl -X DELETE -d '{"name":"flow-mod-1"}'  
http://<controller_ip>:8080/wm/staticflowentrypusher/json
```

6.4 交换机控制器配置

图例 7 中的交换机 1 配置：

Switch1#configure terminal	进入配置视图
Switch1 (config)# openflow set controller tcp 10.10.38.96 6633	设置控制器 IPv4 地址及 TCP 端口号
Switch1 (config)#end	退出配置视图
Switch1# show openflow controller status	显示控制器配置及连接状态

图例 8 中的交换机 1 配置：

Switch1# configure terminal	进入配置视图
Switch1 (config)# openflow set controller tcp 10.10.38.96 6633 controller tcp 10.10.38.96 6634 controller tcp 10.10.38.96 6635	设置多个控制器 IPv4 地址及 TCP 端口号
Switch1 (config)#end	退出配置视图
Switch1# show openflow controller status	显示控制器配置及连接状态

7 配置 Meter

7.1 概述

OpenFlow 中的 Meter 通过测量所有关联到该 Meter 的流表的速率并对速率进行控制。

每个 Meter 可以配置一个丢弃 Band，该 Band 可以指定一个速率，用来限制流量的速率，所有被该 Meter 测量包的流量如果超过了 Band 中配置的速率将会被丢弃。

Meter 的统计对该 Meter 所测量到的流量进行测量，Band 统计则对丢弃的流量进行测量。

在默认流表模式下，V350/V150 支持 2560 个 Meter，所有 Meter 均支持统计。

用户可以通过传统 CLI 方式或 OpenFlow 控制器来配置 Meter、查询 Meter 配置和统计。

7.2 参考资料

- OpenFlow 1.3.1 规格
- RFC 2697
- Open vSwitch

7.3 配置 Meter

V350/V150 支持创建 Meter 并将流表关联到 Meter 上，以便使能流量限制。

在下面的配置示例中，将创建一个 Meter 并关联到流表上。



Figure 7-1 : Meter Topology

Switch

Switch# ovs-ofctl add-meter br0 meter_id=1,flags=KBPS-BURST-STATS,band_type=drop,rate=10000,burst=128 -O openflow13	该 Meter 限制流量速率为 10Mbps, burst 为 128Kbits
Switch# ovs-ofctl add-flow br0 in_port=1,actions=meter:1,output:2 -O openflow13	添加一条流表并关联 Meter

采用如下命令行来查看 Meter 信息，关于命令行的详细信息请参考命令行手册。

Switch# ovs-ofctl dump-meters br0 -O openflow13	显示所有 Meter 配置
Switch# ovs-ofctl dump-meter-stats br0 -O openflow13	显示所有 Meter 统计
Switch# ovs-ofctl dump-meter-feat br0 -O openflow13	显示 Meter 特性支持情况

8 配置 QinQ

8.1 概述

QinQ 技术通过在以太帧中堆叠两个 802.1Q 包头，有效地扩展了 VLAN 数目，使 VLAN 的数目最多可达 4096×4096 个。

V350/V150 支持同时操作两层 VLAN tag，包括：1) 压入/弹出/编辑外层 VLAN tag；2) 压入两层 VLAN tag；3) 剥掉一层 VLAN tag。

8.2 参考资料

- OpenFlow 1.3.1 specification
- Open vSwitch
- IEEE 802.1q
- IEEE 802.1ad

8.3 配置 QinQ

V350/V150 支持同时编辑两层 VLAN tag。

以下示例介绍如何对 VLAN tag 进行编辑。

- Switch1: VLAN tag 16 前插入 VLAN tag 300
- Switch2: 外层 VLAN tag 300 更新为 400
- Switch3: 剥掉 VLAN tag



Figure 8-1 : QinQ 拓扑

Switch1

Switch1# ovs-ofctl add-flow br0 “in_port=1,dl_vlan=16,actions=push_vlan:0x8100,set_field:300 ->vlan_vid,output:2” -O openflow13	插入新的 VLAN tag 300
---	----------------------

Switch2

Switch2# ovs-ofctl add-flow br0 “in_port=1,dl_vlan=300,actions=set_field:400->vlan_vid,output :2” -O openflow13	外层 VLAN tag 300 更 新为 400
---	-----------------------------

Switch3

Switch2# ovs-ofctl add-flow br0 “in_port=1,dl_vlan=400,actions= pop_vlan,output:2” -O openflow13	剥掉 VLAN tag
--	-------------

9 配置 Group

9.1 概述

从 OpenFlow 1.1 开始，Group 特性被引入到 OpenFlow 标准中，流表可以通过指向到 Group 来实现以往指向到端口上所不能的额外功能。

在默认流表模式下，V350/V150 支持 OpenFlow 的全部 Group 类型：

- All: 执行 Group 中的所有 Bucket，这种类型的 Group 用于实现组播或广播。
- Select: 根据报文的 Hash 值（在 V350/V150 上由 ASIC 芯片计算出来）执行 Group 中的一个 bucket，Hash 值的计算算法可以根据需要进行调整。
- Indirect: 执行一个指定的 Bucket，这种类型的 Group 只支持一个 bucket，这个 bucket 只能指向另外的 Group（该 Group 不能也是 Indirect 类型）。
- Fast Failover: 执行首个可转发的 Bucket，每个 Bucket 都会关联一个端口用于检测该 Bucket 的转发状态；可转发的 Bucket 按照 Group 中的 Bucket 定义顺序选择，首个可转发的 Bucket 将被选做用于报文转发；这种类型的 Group 用于在交换机本地切换，避免控制器干预，如果 Group 没有可转发的 Bucket，指定该 Group 的流表将丢弃匹配的报文。

在默认流表模式下，V350/V150 可支持 63 个每种类型的 Group，每个 Group 可以包含 16 个 Bucket（Indirect Group 只能支持 1 个 Bucket）。

用户可通过传统 CLI 方式可 OpenFlow 控制器来配置 Group。

9.2 参考资料

- OpenFlow 1.3.1 specification
- Open vSwitch

9.3 配置 ALL 类型 Group

以下是 ALL 类型 Group 的配置举例。

Switch 1

Switch1# ovs-ofctl add-group br0 group_id=1,type=all,bucket=output:2,bucket=mod_vlan_vid:10,output:3,bucket=mod_dl_src:01:02:03:04:05:06,output:4 -O openflow13	创建 ALL 类型的 group 1，具有三个不同的 Bucket，其中一个 Bucket 将报文不做修改转发到端口 2，一个修改 VLAN 并转发到端口 3，另一个修改源 mac 并转发到端口 4
Switch1# ovs-ofctl add-flow br0 in_port=1,actions=group:1 -O openflow13	创建流表转发到 group 1

配置验证

验证交换机上的 Group 配置。

```
Switch1# ovs-ofctl dump-groups br0 -O openflow13
```

```
OFPST_GROUP_DESC reply (OF1.3) (xid=0x2):
  cookie=0x0, duration=141.256s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=group:1
  group_id=1,type=ALL,bucket=output:2,bucket=mod_vlan_vid:10,output:3,bucket=mod_dl_src:01:02:03:04:05:06,output:4
```

```
Switch1# ovs-ofctl dump-group-stats br0 -O openflow13
```

```
OFPST_GROUP reply (OF1.3) (xid=0x2):
  group_id=1,duration=640.275s,ref_count=1,packet_count=0,byte_count=0
```

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPST_FLOW reply (OF1.3) (xid=0x2):
  cookie=0x0, duration=141.256s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=group:1
```

9.4 配置 SELECT 类型 Group

SELECT 类型 Group 的 bucket 只能支持一个 output 端口（物理端口或 GRE 隧道逻辑端口）。

SELECT 类型的 Group 采用的 hash 算法可以由用户根据应用场景进行配置，详细配置请参考《命令行手册》中的“group select hash key”一节。

注意如果在 bucket 未配置 watch port 的情况下，即便 output 端口状态变为 down，该 bucket 也会继续参与到转发中。

以下是 SELECT 类型 Group 的配置举例。

Switch 1

Switch1# ovs-ofctl add-group br0 group_id=2,type=select,bucket=watch_port:2,output:2,bucket=watch_port:3,output:3,bucket=watch_port:4,output:4 -O openflow13	创建 SELECT 类型的 group 2，具有三个不同的 bucket，分别对报文不做修改转发到端口 2、3、4，每个 bucket 的 watch port 均置为其 output 端口
Switch1# ovs-ofctl add-flow br0 in_port=1,actions=group:2 -O openflow13	创建流表转发到 group 2

配置验证

验证交换机 1 上的 Group 配置。

```
Switch1# ovs-ofctl dump-groups br0 -O openflow13
```

```
OFPST_GROUP_DESC reply (OF1.3) (xid=0x2):  
  
group_id=2, type=SELECT, bucket=watch_port:2, output:2, bucket=watch_port:3, output:3, bucket=watch_port:4, output:4
```

```
Switch1# ovs-ofctl dump-group-stats br0 -O openflow13
```

```
OFPST_GROUP reply (OF1.3) (xid=0x2):  
group_id=2, duration=4.275s, ref_count=1, packet_count=0, byte_count=0
```

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```

OFPST_FLOW reply (OF1.3) (xid=0x2):
  cookie=0x0, duration=4.048s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=group:2

```

Switch1# show group select hash

```

Group select hash use      : mac ip
Group select hash mac key  : port macsa macda vid cos ether-type
Group select hash ip key   : ipsa ipda ip-protocol
Group select hash layer4 key : src-port dst-port

```

9.5 配置 INDIRECT 类型 Group

以下是 INDIRECT 类型 Group 的配置举例。

Switch 1

Switch1# ovs-ofctl add-group br0 group_id=3,type=select,bucket=pop_vlan,watch_port:2,output:2,bucket=pop_vlan,watch_port:3,output:3,bucket=pop_vlan,watch_port:4,output:4 -O openflow13	创建 SELECT 类型的 group 3，具有三个不同的 bucket，分别对报文剥掉 VLAN 后转发到端口 2、3、4，每个 bucket 的 watch port 均置为其 output 端口
Switch1# ovs-ofctl add-group br0 group_id=4,type=indirect, bucket=group:3 -O openflow13	创建 INDIRECT 类型的 group 4，关联到 group 3
Switch1# ovs-ofctl add-flow br0 in_port=1,actions=group:4 -O openflow13	创建流表转发到 group 4

配置验证

验证交换机 1 上的 Group 配置。

Switch1# ovs-ofctl dump-groups br0 -O openflow13

```

OFPST_GROUP_DESC reply (OF1.3) (xid=0x2):

group_id=3,type=SELECT,bucket=watch_port:2,strip_vlan,output:2,bucket=watch_port:3,strip_vlan,output:3,bucket=watch_port:4,strip_vlan,output:4
group_id=4,type=INDIRECT,bucket=group:3

```

```
Switch# ovs-ofctl dump-group-stats br0 -O openflow13
```

```

OFPST_GROUP reply (OF1.3) (xid=0x2):
  group_id=3,duration=54.113s,ref_count=1,packet_count=0,byte_count=0
  group_id=4,duration=30.998s,ref_count=1,packet_count=0,byte_count=0

```

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```

OFPST_FLOW reply (OF1.3) (xid=0x2):
  cookie=0x0, duration=40.932s, table=0, n_packets=0, n_bytes=0, in_port=1
  actions=group:4

```

9.6 配置 FAST FAILOVER 类型 Group

以下是 FAST FAILOVER 类型 Group 的配置举例。

Switch 1

Switch1# ovs-ofctl add-group br0 group_id=5,type=ff,bucket=watch_port:2,output:2,bucket=watch_port:3,mod_vlan_vid:10,output:3,bucket=watch_port:4,mod_dl_src:01:02:03:04:05:06,output:4 -O openflow13	配置 FAST FAILOVER 类型的 group 5，具有三个 bucket，其中一个 bucket 将报文不做修改转发给端口 2，一个 bucket 将报文修改 VLAN 后转发给端口 3，一个修改源 mac 后转发给端口 4
Switch1# ovs-ofctl add-flow br0 in_port=1,actions=group:5 -O openflow13	创建流表转发到 group 5

配置验证

验证交换机 1 上的 Group 配置。

```
Switch1# ovs-ofctl dump-groups br0 -O openflow13
```

```

OFPST_GROUP_DESC reply (OF1.3) (xid=0x2):
  group_id=5,type=FF,bucket=watch_port:2,output:2,bucket=watch_port:3,mod_vlan_vid:10,output:3,bucket=watch_port:4,mod_dl_src:01:02:03:04:05:06,output:4

```

Switch1# ovs-ofctl dump-group-stats br0 -O openflow13

```
OFPST_GROUP reply (OF1.3) (xid=0x2):  
group_id=5, duration=34.751s, ref_count=1, packet_count=0, byte_count=0
```

Switch1# ovs-ofctl dump-flows br0 -O openflow13

```
OFPST_FLOW reply (OF1.3) (xid=0x2):  
cookie=0x0, duration=44.704s, table=0, n_packets=0, n_bytes=0, in_port=1  
actions=group:
```

10 配置 QoS

10.1 概述

V350/V150 支持 OpenFlow 中的 Queue 机制，用户可以使用该机制对不同类型的流量进行调度，实现简单的差分服务。

在默认流表模式下，V350/V150 每个端口支持 8 个 queue，并提供配置最小或最大的带宽保证。

10.2 参考资料

- OpenFlow 1.3.1 specification
- Open vSwitch

10.3 配置 Queue 整形

以下示例介绍如何配置 Queue 并配置流表关联到 Queue。

Switch

Switch# ovs-vsctl set-queue-config eth-0-2 0 30 60	配置端口 eth-0-2 上 queue 0 的最小速率为端口速率的千分之 30，最大速率为端口速率的千分之 60
Switch# ovs-ofctl add-flow br0 " in_port=1, actions=set_queue:0,output:2"	添加流表，将端口 1 上收到的报文送入端口 2 的 queue 0 转发出去
Switch(config)# ipg queue enable	使能 IPG

采用如下命令显示 Queue 配置及 Queue 统计。

Switch# ovs-ofctl dump-queue-config br0 2 -O OPENFLOW13	显示端口 2 的 Queue 配置
Switch# ovs-ofctl dump-queue-stats br0 2 -O OPENFLOW13	显示端口 2 的 Queue 统计

验证配置

验证交换机上的 Queue 配置。

Switch1# ovs-ofctl dump-queue-config br0 2 -O OPENFLOW13

```
OFPT_QUEUE_GET_CONFIG_REPLY (OF1.3) (xid=0x2): 8 queues
port 2 queue 0: min=30, max=60
port 2 queue 1: min=?, max=?
port 2 queue 2: min=?, max=?
port 2 queue 3: min=?, max=?
port 2 queue 4: min=?, max=?
port 2 queue 5: min=?, max=?
port 2 queue 6: min=?, max=?
port 2 queue 7: min=?, max=?
```

Switch# ovs-ofctl dump-queue-stats br0 2 -O OPENFLOW13

```
OFPT_QUEUE_REPLY (OF1.3) (xid=0x2): 8 queues
port 2 queue 0: bytes=0, pkts=0, errors=0, duration_sec=13783, duration_nsec=882000000
port 2 queue 1: bytes=0, pkts=0, errors=0, duration_sec=13783, duration_nsec=882000000
port 2 queue 2: bytes=0, pkts=0, errors=0, duration_sec=13783, duration_nsec=882000000
port 2 queue 3: bytes=0, pkts=0, errors=0, duration_sec=13783, duration_nsec=883000000
port 2 queue 4: bytes=0, pkts=0, errors=0, duration_sec=13783, duration_nsec=883000000
port 2 queue 5: bytes=0, pkts=0, errors=0, duration_sec=13783, duration_nsec=883000000
port 2 queue 6: bytes=0, pkts=0, errors=0, duration_sec=13783, duration_nsec=883000000
port 2 queue 7: bytes=0, pkts=0, errors=0, duration_sec=13783, duration_nsec=883000000
```

Switch1# ovs-ofctl dump-flows br0 -O openflow13

```
OFPT_FLOW_REPLY (OF1.3) (xid=0x2):
cookie=0x0, duration=95.073s, table=0, n_packets=14989630208, n_bytes=234212972,
in_port=1 actions=set_queue:0,output:2
```

10.4 配置 Classification

流分类指采用一定的规则识别出符合某类特征的报文。报文进入系统时，流分类处理引擎会为报文分配一个内部优先级，基于这个优先级，系统对报文进行一系列的处理。V350/V150 支持通过基于报文的 TOS 字段将报文映射到不同队列。

以下示例介绍如何配置 classification。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# interface eth-0-2	进入接口视图
Switch(config-if)# trust tos	设置 trust 状态
Switch(config-if)# exit	退出到配置视图
Switch(config)# qos map tos-queue-id 128 to 7	配置 TOS 128 映射到 queue 7

采用如下命令显示 trust 状态。

Switch# show qos interface eth-0-2	显示端口 eth-0-2 上的配置
Switch# show qos map-table tos-queue-id running	显示 qos 映射表信息

验证配置

验证交换机上的 Classification 配置。

Switch# show qos interface eth-0-2

```
Interface trust state: tos

Schedule mode: SP(between Class), WDRR(between queue in the same Class)
The number of class on interface: 7
Strict priority class ID: 6 5 4 3 2 1 0
The number of egress queue: 8
Queue 0 class 0
  Tail drop mode
  Transmit 0 packets, 0 bytes
  Drop 0 packets, 0 bytes
Queue 1 class 0
  Tail drop mode
  Transmit 0 packets, 0 bytes
  Drop 0 packets, 0 bytes
Queue 2 class 1
  Tail drop mode
  Transmit 0 packets, 0 bytes
  Drop 0 packets, 0 bytes
```

```

Queue 3 class 1
  Tail drop mode
  Transmit 0 packets, 0 bytes
  Drop 0 packets, 0 bytes
Queue 4 class 2
  Tail drop mode
  Transmit 0 packets, 0 bytes
  Drop 0 packets, 0 bytes
Queue 5 class 2
  Tail drop mode
  Transmit 0 packets, 0 bytes
  Drop 0 packets, 0 bytes
Queue 6 class 3
  Tail drop mode
  Transmit 0 packets, 0 bytes
  Drop 0 packets, 0 bytes
Queue 7 class 3
  Tail drop mode
  Transmit 0 packets, 0 bytes
  Drop 0 packets, 0 bytes

```

Switch# show qos map-table tos-queue-id running

```

QoS TOS map to Queue-ID:
-----
TOS      :   0   1   2   3   4   5   6   7
Queue-id :   0   0   0   0   0   0   0   0
TOS      :   8   9  10  11  12  13  14  15
Queue-id :   0   0   0   0   0   0   0   0
TOS      :  16  17  18  19  20  21  22  23
Queue-id :   0   0   0   0   0   0   0   0
TOS      :  24  25  26  27  28  29  30  31
Queue-id :   0   0   0   0   0   0   0   0
TOS      :  32  33  34  35  36  37  38  39
Queue-id :   1   1   1   1   1   1   1   1
TOS      :  40  41  42  43  44  45  46  47
Queue-id :   1   1   1   1   1   1   1   1
TOS      :  48  49  50  51  52  53  54  55
Queue-id :   1   1   1   1   1   1   1   1
TOS      :  56  57  58  59  60  61  62  63
Queue-id :   1   1   1   1   1   1   1   1
TOS      :  64  65  66  67  68  69  70  71
Queue-id :   2   2   2   2   2   2   2   2
TOS      :  72  73  74  75  76  77  78  79
Queue-id :   2   2   2   2   2   2   2   2
TOS      :  80  81  82  83  84  85  86  87
Queue-id :   2   2   2   2   2   2   2   2
TOS      :  88  89  90  91  92  93  94  95
Queue-id :   2   2   2   2   2   2   2   2
TOS      :  96  97  98  99 100 101 102 103
Queue-id :   3   3   3   3   3   3   3   3
TOS      : 104 105 106 107 108 109 110 111
Queue-id :   3   3   3   3   3   3   3   3
TOS      : 112 113 114 115 116 117 118 119
Queue-id :   3   3   3   3   3   3   3   3

```

```

TOS      : 120 121 122 123 124 125 126 127
Queue-id :   3   3   3   3   3   3   3   3
TOS      : 128 129 130 131 132 133 134 135
Queue-id :   7   4   4   4   4   4   4   4
TOS      : 136 137 138 139 140 141 142 143
Queue-id :   4   4   4   4   4   4   4   4
TOS      : 144 145 146 147 148 149 150 151
Queue-id :   4   4   4   4   4   4   4   4
TOS      : 152 153 154 155 156 157 158 159
Queue-id :   4   4   4   4   4   4   4   4
TOS      : 160 161 162 163 164 165 166 167
Queue-id :   5   5   5   5   5   5   5   5
TOS      : 168 169 170 171 172 173 174 175
Queue-id :   5   5   5   5   5   5   5   5
TOS      : 176 177 178 179 180 181 182 183
Queue-id :   5   5   5   5   5   5   5   5
TOS      : 184 185 186 187 188 189 190 191
Queue-id :   5   5   5   5   5   5   5   5
TOS      : 192 193 194 195 196 197 198 199
Queue-id :   6   6   6   6   6   6   6   6
TOS      : 200 201 202 203 204 205 206 207
Queue-id :   6   6   6   6   6   6   6   6
TOS      : 208 209 210 211 212 213 214 215
Queue-id :   6   6   6   6   6   6   6   6
TOS      : 216 217 218 219 220 221 222 223
Queue-id :   6   6   6   6   6   6   6   6
TOS      : 224 225 226 227 228 229 230 231
Queue-id :   7   7   7   7   7   7   7   7
TOS      : 232 233 234 235 236 237 238 239
Queue-id :   7   7   7   7   7   7   7   7
TOS      : 240 241 242 243 244 245 246 247
Queue-id :   7   7   7   7   7   7   7   7
TOS      : 248 249 250 251 252 253 254 255
Queue-id :   7   7   7   7   7   7   7   7

```

10.5 配置 Scheduling

系统为每个队列分配一个优先级（class），范围是 0 到 6，数字越高表示优先级越高。端口上的 8 个队列的优先级是可配置的。一个端口上，不同的优先级之间使用的 SP 调度，即高优先级队列先被调度，当高优先级队列为空时才会调度低优先级队列。相同的优先级内的队列采用 WDRR 调度。用户可以为各个队列设置 WDRR 权重。

- **Strict Priority-Based (SP)**, 严格优先级，按照优先级从高到低的次序优先发送较高优先级队列中的报文，当较高优先级队列为空时，再发送较低优先级队列中的报文。
- **Weighted Deficit Round Robin (WDRR)**, 加权差额循环调度，每个队列一个权重。配额代表权重，以字节为单位，是可配置参数，余额用来表示配额的积累和消耗情况，是状态参数，不可配置。初始状态下，每个队列的余额等于其配额，当队列每发送一个报文的时候，余额就减去这个报文的字节数，当余额低于 0 的时候就停止这个队列的调度。当所有队列都停止调度的时候就为所有队列补充配额。

以下示例介绍如何配置 scheduling。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# interface eth-0-2	进入端口 eth-0-2 的接口视图
Switch(config-if)# queue 2 class 0	设置 queue 2 的优先级为 0
Switch(config-if)# queue 3 class 0	设置 queue 3 的优先级为 0
Switch(config-if)# queue 6 class 6	设置 queue 6 的优先级为 6
Switch(config-if)# queue 7 class 6	设置 queue 7 的优先级为 6

采用如下命令显示 Queue 配置。

Switch# show qos interface eth-0-2	显示端口 2 的 Queue 配置
------------------------------------	-------------------

验证配置

验证交换机上的 Queue 配置。

Switch1# show qos interface eth-0-2

```
Interface trust state: tos

Schedule mode: SP(between Class), WRR(between queue in the same Class)
The number of class on interface: 7
Strict priority class ID: 6 5 4 3 2 1 0
The number of egress queue: 8
Queue 0 class 0
  Tail drop mode
  Transmit 0 packets, 0 bytes
  Drop 0 packets, 0 bytes
Queue 1 class 0
  Tail drop mode
  Transmit 0 packets, 0 bytes
  Drop 0 packets, 0 bytes
Queue 2 class 0
  Tail drop mode
  Transmit 0 packets, 0 bytes
  Drop 0 packets, 0 bytes
Queue 3 class 0
  Tail drop mode
  Transmit 0 packets, 0 bytes
```

```
Drop 0 packets, 0 bytes
Queue 4 class 2
Tail drop mode
Transmit 0 packets, 0 bytes
Drop 0 packets, 0 bytes
Queue 5 class 2
Tail drop mode
Transmit 0 packets, 0 bytes
Drop 0 packets, 0 bytes
Queue 6 class 6
Tail drop mode
Transmit 0 packets, 0 bytes
Drop 0 packets, 0 bytes
Queue 7 class 6
Tail drop mode
Transmit 0 packets, 0 bytes
Drop 0 packets, 0 bytes
```

11 配置 sFlow

11.1 概述

sFlow 是一种网络检测技术，用户可以在 sFlow 代理上配置不同采样方法对网络流量进程监控，sFlow 代理不断发送采样数据报文给 sFlow 采集器，采集器对这些数据报文进行分析，让用户详细、实时地分析网络传输流的性能、趋势和存在的问题。

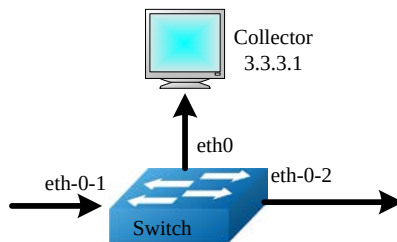
sFlow 使用两种独立的采样方法来获取数据，针对数据流的基于数据包的统计采样方法和针对网络接口基于时间的统计采样方法。

11.2 参考资料

- OpenFlow 1.3.1 specification
- Open vSwitch

11.3 配置 sFlows

在下面的配置示例中，假设 sflow 使用 UDP 端口 6343，发送检测报文到收集器 3.3.3.1。

**Figure 11-1 : sFlow 拓扑**

Switch

Switch# ovs-vsctl -- --id=@s create sFlow agent=eth0 target="3.3.3.1:6343" header=128 sampling=400 polling=10 -- set Bridge br0 sflow=@s	创建 sflow，收集器为 3.3.3.1，代理端口 为带外管理口 eth0，采样速率 400（即每 400 个报文中采样一个），轮询间隔 10 秒。
---	---

以下命令验证 sflow 的配置结果，更多命令细节可参考命令行相关文档。

Switch# show sflow	显示 sflow 配置信息
Switch# ovs-vsctl -- clear Bridge br0 sflow	清除 sFlow 配置。

验证配置

验证交换机上的 sflow 配置：

Switch1# show sflow

```
sFlow Global Information:
Agent Device           : eth0
Agent IP address       : 10.10.38.107
Counter Sampling Interval : 10 seconds
Packets Sampling Rate  : 400
Header Length          : 128 bytes
Collector IP           : 3.3.3.1      Port: 6343
```

Switch# ovs-vsctl list sflow

```
_uuid      : 4e0a04b8-f743-4fc7-972b-f026e8ce6333
agent      : "eth0"
```

```
external_ids      : {}  
header           : 128  
polling          : 10  
sampling         : 400  
targets          : ["3.3.3.1:6343"]
```

12 配置 Bond

12.1 概述

V350/V150 支持 Bond 端口，用来实现链路绑定或链路聚合，从而提高与接入 V350/V150 设备间的带宽或实现主备链路保护。

用户可以通过 CLI 静态配置，或利用 OVSDB 协议远程配置来配置 Bond 接口，这些接口需要以 “bond1”、“bond2” 这样的形式命名，Bond 端口的 OpenFlow 逻辑端口号以 401 开始，该逻辑端口可用于流表的入端口匹配或出端口转发。

在默认流表模式下，V350/V150 可最大支持 16 个 Bond 端口，每个 Bond 端口最大支持 16 成员。

12.2 配置 Bond

在下面的配置示例中，在 V350/V150 交换机上创建 Bond 端口 bond1，其成员口为物理端口 eth-0-1 和端口 eth-0-2。

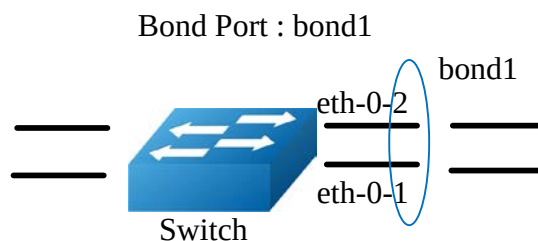


Figure 12-1 : Bond port 拓扑

Switch

Switch# ovs-vsctl del-port br0 eth-0-1	从 br0 中删除物理端口 eth-0-1
Switch# ovs-vsctl del-port br0 eth-0-2	从 br0 中删除物理端口 eth-0-2
Switch# ovs-vsctl add-bond br0 bond1 eth-0-1 eth-0-2 bond_mode=balance-slb -- set interface eth-0-1 type=switch -- set interface eth-0-2 type=switch	创建负载分担类型的 bond1 端口，成员口为物理端口 eth-0-1 和物理端口 eth-0-2
Switch1# ovs-ofctl add-flow br0 "ip,action=output:401"	添加一条流表转发到 bond1

说明

- 1、在将物理端口加入 Bond 端口前，需要先将物理端口从 br0 中删除。
- 2、V350/V150 支持两种 bond_mode，分别为负载分担模式和主备模式，在配置前者时请使用 balance-slb 参数，配置后者时请使用 active-backup 参数。

配置验证

验证交换机上的 Bond 配置：

Switch1# **show interface bond brief**

The Maximum of bond port is 16, currently 1 bond ports is valid.

```
---- bond1 ----
bond_mode  : balance-slb
bond_id    : 1
bond_ofport: 401
bond_ref   : 0

slave eth-0-1: enabled
slave eth-0-2: enabled
```

Switch1# **ovs-ofctl dump-flows br0 -O openflow13**

```
OFPT_FLOW reply (OF1.3) (xid=0x2):
 cookie=0x0, duration=3.252s, table=0, n_packets=0, n_bytes=0, ip actions=output:401
```

12.3 配置 Bond Hash 参数

用户可以使用“bond select hash”来全局控制流量在负载分担类型 Bond 端口上的负载分担算法。

Switch

Switch (config)# bond select hash use mac	使用 mac 头进行负载分担计算
Switch (config)# bond select hash mac macsa macda	使用源 mac 和目的 mac 做为 mac 头负载分担 Hash 计算因子。

配置验证

验证交换机上 Bond Hash 计算配置：

Switch1# **show bond select hash**

```
bond select hash use      : mac
bond select hash mac key  : macsa macda
bond select hash ip key   : ipsa ipda ip-protocol
bond select hash layer4 key : src-port dst-port
```

13 配置二层 hash 模式

13.1 概述

二层 hash 流表模式是盛科为满足部分客户对大容量二层 hash 流表的需求而定制的专用流表模式，该模式支持 16000 的条二层精确匹配流表。

流表数量	16000
Wildcard 匹配字段	不支持
精确匹配字段	匹配字段必须是以下字段的一种或几种： IN_PORT MAC_SA/MAC_DA/VLAN_ID/VLAN_PCP
流表行为	OUTPUT SET_FIELD(MAC_SA/MAC_DA/VLAN_ID/VLAN_PCP) PUSH_VLAN/POP_VLAN
Group	不支持
MPLS	不支持
GRE	不支持
QinQ	支持
Meter	支持
Queue	不支持
sFlow	支持

13.2 参考资料

- OpenFlow 1.3.1 specification
- Open vSwitch

13.3 配置二层 hash 流表模式

以下示例介绍如何切换交换机到二层 hash 流表模式。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# stm prefer l2hash	切换到二层 hash 流表模式
Switch(config)# end	退出到特权模式
Switch# show stm prefer	显示当前和下一次重启使用的流表模式
Switch# reboot	重启系统以便新的流表模式生效

验证配置

验证交换机上的二层 hash 流表模式配置（尚未重启）：

Switch1# show stm prefer

```
Current profile is : default
Next profile is   : l2hash
```

13.4 配置二层 hash key

以下示例介绍如何配置二层 hash 流表模式中 hash 算法使用到的 key。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# flow l2hash mac port macsa macda	配置二层 hash 流表模式下的 hash key(port macsa macda)
Switch(config)# end	退出到特权视图

Switch# show flow l2hash	查看二层 hash 流表模式中配置的 hash key
--------------------------	-----------------------------

验证配置

验证交换机上的二层 hash 流表模式的 hash key 配置：

```
Switch1# show flow l2hash
```

```
Flow l2hash mac key      : port macsa macda
```

13.5 配置二层 hash 流表

以下示例介绍如何在二层 hash 流表模式下配置流表。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# flow l2hash mac port macsa macda	配置二层 hash 流表模式下的 hash key(port macsa macda)
Switch(config)# end	退出到特权视图
Switch# ovs-ofctl add-flow br0 priority=0xffff,in_port=1,dl_src=00:01:02:03:04:05,dl_dst=00:06:07:08:09:0a,actions=output:2 -O openflow13	添加二层 hash 流表
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

验证配置

验证交换机上的二层 hash 流表配置：

```
Switch# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPT_FLOW reply (OF1.3) (xid=0x2):
 cookie=0x0, duration=9.162s, table=0, n_packets=0, n_bytes=0,
 priority=65535,in_port=1,dl_src=00:01:02:03:04:05,dl_dst=00:06:07:08:09:0a
 actions=output:2
```

14 配置三层 hash 流表模式

14.1 概述

三层 Hash 流表模式是盛科为满足部分客户对大容量三层 hash 流表的需求而定制的专用流表模式，该模式支持 16000 条的三层 hash 精确匹配流表。

流表数量	32000
Wildcard 匹配字段	不支持
精确匹配字段	匹配字段必须是以下字段的一种或几种： IN_PORT/ETHER_TYPE IPSA/IPDA/IP_DSCP/L3_PROTOCOL/ TCP_UDP_SRC_PORT/TCP_UDP_DST_PORT
流表行为	OUTPUT SET_FIELD(MAC_SA/MAC_DA/ VLAN_VID/VLAN_PCP/IPV4_DA/TCP_DST_PORT/UDP_DS T_PORT) PUSH_VLAN/POP_VLAN
Group	不支持
MPLS	不支持
GRE	不支持
QinQ	支持
Meter	不支持

Queue	不支持
sFlow	支持

14.2 参考资料

- OpenFlow 1.3.1 specification
- Open vSwitch

14.3 配置 l3hash 流表模式

以下示例介绍如何切换交换机到三层 hash 流表模式。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# stm prefer l3hash	切换到三层 hash 流表模式
Switch(config)# end	退出到特权视图
Switch# show stm prefer	显示当前和下一次重启使用的流表模式
Switch# reboot	重启系统以便新的流表模式生效

验证配置

验证交换机上的三层 hash 流表模式配置（尚未重启）：

```
Switch1# show stm prefer
```

```
Current profile is : default
Next profile is    : l3hash
```

14.4 配置三层 hash key

以下示例介绍如何配置三层 hash 流表模式中 hash 算法使用到的 key。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# flow l3hash ip flow l3hash ip port ipsa ipda dscp ip-protocol src-port dst-port	配置三层 hash 流表模式下的 hash key (port, ipsa, ipda, dscp, ip-protocol, src-port, dst-port)
Switch(config)# end	退出到特权视图
Switch# show flow l3hash	查看三层 hash 流表模式中配置的 hash key

验证配置

验证交换机上 hash key 的配置：

Switch1# show flow l3hash

```
Flow l3hash mac key : port ipsa ipda dscp ip-protocol src-port dst-por
```

14.5 配置三层 hash 流表

以下示例介绍如何在三层 hash 流表模式下配置流表。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# flow l3hash ip flow l3hash ip port ipsa ipda dscp ip-protocol src-port dst-port	配置三层 hash 流表模式下的 hash key (port, ipsa, ipda, dscp, ip-protocol, src-port, dst-port)
Switch(config)# end	退出到特权视图
Switch# ovs-ofctl add-flow br0 "priority=65535,tcp,in_port=1,nw_src=1.12 .23.34,nw_dst=45.56.67.78,ip_dscp=5,tp_sr c=12,tp_dst=23 actions=output:3" -Oopenflow13	添加三层 hash 流表
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

验证配置

验证交换机上的三层 hash 流表配置：

Switch# ovs-ofctl dump-flows br0 -O openflow13

```
OFPST_FLOW reply (OF1.3) (xid=0x2):  
  cookie=0x0, duration=11.973s, table=0, n_packets=0, n_bytes=0, priority=65535,tcp,  
  in_port=1,nw_src=1.12.23.34,nw_dst=45.56.67.78, ip_dscp=5,tp_src=12,tp_dst=23  
  actions=output:3
```

15 配置 NAT 模式

15.1 概述

NAT 流表模式是盛科为 NAT 应用定制的专用流表模式，在配置 NAT 流表模式后，V350/V150 默认工作在 5 元组子模式。

子模式	5 元组
	SNAT/DNAT
流表数量	2000
Wildcard 匹配字段	IN_PORT IPDA IPSA TCP/UDP SRC_PORT DST_PORT
精确匹配 字段	N/A

流表行为	OUTPUT POR_VLAN PUSH_VLAN SET_FIELD(VLAN_VID IPV4_DA IPV4_SA MAC_DA MAC_SA SRC_PORT DST_PORT)
Group	支持
Stats	支持
MPLS	不支持
GRE	不支持
QinQ	不支持
Meter	不支持
Queue	不支持
sFlow	不支持

15.2 参考资料

- OpenFlow 1.3.1 标准
- Open vSwitch

15.3 配置 NAT 流表模式

以下示例介绍如何切换交换机到 NAT 流表模式。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# stm prefer nat	切换到 NAT 流表模式
Switch(config)# end	退出到特权视图
Switch# show stm prefer	显示当前和下一次重启使用的流表模式
Switch# reboot	重启系统以便新的流表模式生效

配置验证

验证交换机上的 NAT 流表模式配置（尚未重启）：

```
Switch1# show stm prefer
```

```
Current profile is : default
Next profile is    : nat
```

15.4 在 5 元组子模式下配置流表

以下示例介绍如何在 5 元组子模式下配置流表。

Switch

Switch# ovs-ofctl add-flow br0 "priority=65535,tcp,in_port=1,ip,nw_src=1.1.1.1,nw_dst=3.3.3.3,tp_src=3000,actions=mod_dl_src:2e:4a:2e:52:0a:00,mod_nw_src:2.2.2.2,output:2" -O openflow13	配置 SNAT 流表
Switch# ovs-ofctl add-flow br0 "priority=65535,tcp,in_port=2,ip,nw_src=3.3.3.3,nw_dst=2.2.2.2,tp_dst=4000,actions=mod_dl_dst:00:13:32:ac:0a:00,mod_nw_dst:1.1.1.1,output:1" -O openflow13	配置 DNAT 流表

Switch# ovs-ofctl add-flow br0 "priority=0,actions=controller" -O openflow13	添加 table miss 流表，将匹配不到任何流表的报文送给控制器
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

配置验证

验证交换机上的 NAT 流表配置：

Switch# ovs-ofctl dump-flows br0 -O openflow13

```
NXST_FLOW reply (xid=0x4):
  cookie=0x0, duration=10.547s, table=0, n_packets=0, n_bytes=0, priority=65535, tcp,in_port=1,nw_src=1.1.1.1,nw_dst=3.3.3.3,tp_src=3000 actions=mod_dl_src:2e:4a:2e:52:0a:00,mod_nw_src:2.2.2.2,output:2
  cookie=0x0, duration=9.776s, table=0, n_packets=0, n_bytes=0, priority=0 actions=CONTROLLER:65535
  cookie=0x0, duration=10.466s, table=0, n_packets=0, n_bytes=0, priority=65535, tcp,in_port=2,nw_src=3.3.3.3,nw_dst=2.2.2.2,tp_dst=4000 actions=mod_dl_dst:00:13:32:ac:0a:00,mod_nw_dst:1.1.1.1,output:1
```

15.5 配置 DNAT 流表的负载分担

以下示例介绍如何配置 DNAT 流表的负载分担。

Switch

Switch# ovs-ofctl add-group br0 "type=select,group_id=1, bucket=mod_nw_dst:2.2.2.2,mod_tp_dst=2 000,watch_port:2,output=2,bucket=mod_n w_dst:3.3.3.3,mod_tp_dst=3000,watch_port :3,output=3" -O openflow13	添加 SELECT 类型的 Group
Switch# ovs-ofctl add-flow br0 "priority=65535,in_port=1,ip,nw_dst=1.1.1. 1,actions=group:1" -O openflow13	添加 DNAT 流表
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

配置验证

验证交换机上的 NAT 流表配置。

```
Switch# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPST_FLOW reply (OF1.3) (xid=0x2):  
  cookie=0x0, duration=6.963s, table=0, n_packets=0, n_bytes=0,  
  priority=65535, ip,in_port=1,nw_dst=1.1.1.1 actions=group:1
```

```
Switch# ovs-ofctl dump-groups br0 -O openflow13
```

```
OFPST_GROUP_DESC reply (OF1.3) (xid=0x2):  
  group_id=1,type=SELECT,bucket=watch_port:2,mod_nw_dst:2.2.2.2,mod_tp_dst:2000,output:  
  t:2,bucket=watch_port:3,mod_nw_dst:3.3.3.3,mod_tp_dst:3000,output:3
```

16 配置 NVCloud 模式

16.1 概述

NVCloud 流表模式是盛科为网络虚拟化应用定制的专用流表模式,该模式支持4张流表: table0 为租户识别表, table1 为信息过滤表, table2 为二层转发表, table3 为三层转发表。 NVCloud 流表模式支持两种租户识别方式, vlan 识别和端口 mac 识别。

该模式下,用户可以根据需求配置采用 L2oGRE 或者 NVGRE 格式对以太报文进行封装。

Table	Tenant identify		Filter	L2FWD	L3FWD
流表数量	local	remote	200	32000	8000
	4000	2000			
Wildcard 匹配字段	不支持		IN_PORT MAC_SA/MAC_DA VLAN_ID/VLAN_PC P ETHER_TYPE ARP_OP/ARP_SPA/ ARP_TPA IPSA/IPDA/IP_DSCP L3_PROTOCOL TCP_SRC_PORT/TC P_DST_PORT UDP_SRC_PORT/U DP_DST_PORT	不支持	不支持

精确匹配 字段	IN_PORT MACSA VLAN_VID	N/A	METADAT A MACDA	METADAT A IPDA
流表行为	WRITE_METADAT A GOTO TABLE	OUTPUT GOTO TABLE STRIP_VLAN PUSH_VLAN SET_FIELD(MACDA/MACSA/TUNNEL_ID/VLAN_VID)	OUTPUT STRIP_VLAN PUSH_VLAN SET_FIELD(MACDA/MACSA/TUNNEL_ID/VLAN_VID)	OUTPUT STRIP_VLAN PUSH_VLAN SET_FIELD(MACDA/MACSA/TUNNEL_ID/VLAN_VID)
Group	不支持			
MPLS	不支持			
GRE	支持			
QinQ	不支持			
Meter	支持(仅限 filter table)			
Queue	不支持			
sFlow	不支持			

16.2 参考资料

- OpenFlow 1.3.1 specification
- Open vSwitch

16.3 配置 nvcloud 流表模式

以下示例介绍如何切换交换机到 nvcloud 流表模式。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# stm prefer nvcloud	切换到 nvcloud 流表模式
Switch(config)# end	退出到特权视图
Switch# show stm prefer	显示当前和下一次重启使用的流表模式
Switch# reboot	重启系统以便新的流表模式生效

验证配置

验证交换机上的 nvcloud 流表模式配置（尚未重启）。

Switch1# show stm prefer

```
Current profile is : default
Next profile is   : nvcloud
```

16.4 配置 nvgre compatible

以下示例介绍如何配置 nvgre compatible。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# nvgre compatible enable	使能 NVGRE compatible
Switch(config)# end	退出到特权视图

验证配置

验证交换机上 nvgre compatible 配置。

Switch1# show debug flow-entry-num

```
NVGRE Compatible enable : Yes
```

16.5 配置租户识别类型

以下示例介绍如何配置租户识别类型，缺省值 vlan。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# tenant identify type vlan	Tenant identified by vlan
Switch(config)# end	退出到特权视图
Switch# ovs-ofctl add-flow br0 "table=0,priority=65535,dl_vlan=10,actions =write_metadata:10,goto_table:1" -O openflow13	添加带 vlan 的流表
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

验证配置

验证交换机上租户识别类型配置。

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

```

OFPST_FLOW reply (OF1.3) (xid=0x2):
  cookie=0x0, duration=9.816s, table=0, n_packets=0, n_bytes=0,
  priority=65535,dl_vlan=10 actions=write_metadata:0xa,goto_table:1

```

Switch

Switch# configure terminal	进入配置视图
Switch(config)# tenant identify type port-mac	配置租户识别类型为 port-mac
Switch(config)# end	退出到特权视图
Switch# ovs-ofctl add-flow br0 "table=0,priority=65535,in_port=1,dl_src=f a:16:3e:11:11:11,actions=write_metadata:1 0,goto_table:1" -O openflow13	添加带端口和源 mac 的流表
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

验证配置

验证交换机上的 nvcloud 流表配置。

```
Switch1# ovs-ofctl dump-flows br0 -O openflow13
```

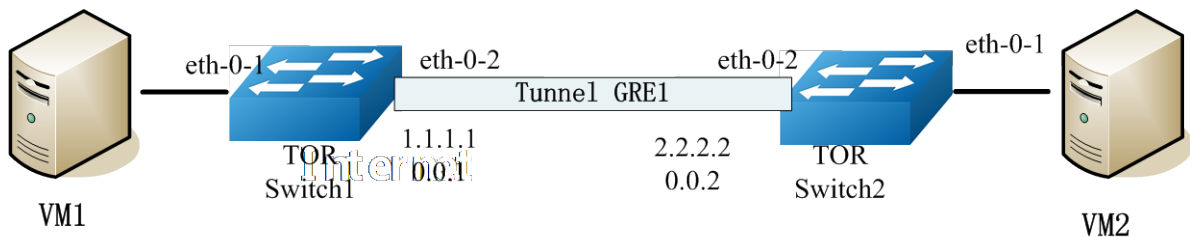
```
OFPST_FLOW reply (OF1.3) (xid=0x2):  
  cookie=0x0, duration=35.288s, table=0, n_packets=0, n_bytes=0,  
  priority=65535,in_port=1,dl_src=fa:16:3e:11:11:11  
  actions=write_metadata:0xa,goto_table:1
```

16.6 配置 L2 forward flow

以下示例介绍如何配置 l2forward 流表。

MAC: fa:16:3e:11:11:11
IP: 192.168.10.1

MAC: fa:16:3e:22:22:22
IP: 192.168.10.2



Switch1

Switch(config)# tenant identify type port-mac	配置租户识别模式为 port-mac
Switch# ovs-vsctl add-port br0 gre1 -- set interface gre1 type=gre options:remote_ip=2.2.2.2 options:local_ip=1.1.1.1 options:bind_port=eth-0-2 options:nexthop_mac=0000.0000.0002	创建 gre 端口 gre1, 本地 ip 为 1.1.1.1, 远端 IP 为 2.2.2.2, 下一跳 mac 为 0.0.2, 绑定端口 eth-0-2
Switch# ovs-ofctl add-flow br0 "table=0,priority=65535,in_port=1,dl_src=fa:16:3e:11:11:11,actions=write_metadata:10,goto_table:1" -O openflow13	为 VM1 到 VM2 的报文添加租户识别流表
Switch# ovs-ofctl add-flow br0 "table=0,priority=65535,tun_id=10,actions=write_metadata:10,goto_table:1" -O openflow13	为 VM2 到 VM1 的报文添加租户识别流表

Switch# ovs-ofctl add-flow br0 "table=1,priority=1,actions=goto_table:2" -O openflow13	添加过滤流表
Switch# ovs-ofctl add-flow br0 "table=2,priority=65535,metadata=10,dl_dst=fa:16:3e:22:22:22,actions=set_tunnel:10,output:201" -O openflow13	为 VM1 到 VM2 的报文添加二层转发流表
Switch# ovs-ofctl add-flow br0 "table=2,priority=65535,metadata=10,dl_dst=fa:16:3e:11:11:11,actions=output:1" -O openflow13	为 VM2 到 VM1 的报文添加二层转发流表，解封装后送到 eth-0-1
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

验证配置

验证交换机上的 nvcloud 流表配置。

Switch1# ovs-ofctl dump-flows br0 -O openflow13

```

OFPST_FLOW reply (OF1.3) (xid=0x2):
  cookie=0x0, duration=38.514s, table=0, n_packets=0, n_bytes=0,
  priority=65535,in_port=1,dl_src=fa:16:3e:11:11:11
  actions=write_metadata:0xa,goto_table:1
  cookie=0x0, duration=29.507s, table=0, n_packets=0, n_bytes=0,
  priority=65535,tun_id=0xa actions=write_metadata:0xa,goto_table:1
  cookie=0x0, duration=22.837s, table=1, n_packets=0, n_bytes=0, priority=1
  actions=goto_table:2
  cookie=0x0, duration=13.856s, table=2, n_packets=?, n_bytes=?,
  priority=65535,metadata=0xa,dl_dst=fa:16:3e:22:22:22
  actions=set_tunnel:0xa,output:201
  cookie=0x0, duration=6.704s, table=2, n_packets=?, n_bytes=?,
  priority=65535,metadata=0xa,dl_dst=fa:16:3e:11:11:11 actions=output:1

```

Switch2

Switch(config)# tenant identify type port-mac	配置租户识别模式为 port-mac
---	-----------------------

Switch# ovs-vsctl add-port br0 gre1 -- set interface gre1 type=gre options:remote_ip=1.1.1.1 options:local_ip=2.2.2.2 options:bind_port=eth-0-2 options:nexthop_mac=0000.0000.0001	创建 gre 端口 gre1，本地 ip 为 2.2.2.2，远端 IP 为 1.1.1.1，下一跳 mac 为 0.0.1，绑定端口 eth-0-2
Switch# ovs-ofctl add-flow br0 "table=0,priority=65535,in_port=1,dl_src=fa:16:3e:22:22:22,actions=write_metadata:10,goto_table:1" -O openflow13	为 VM2 到 VM1 的报文添加租户识别流表
Switch# ovs-ofctl add-flow br0 "table=0,priority=65535,tun_id=10,actions=write_metadata:10,goto_table:1" -O openflow13	为 VM1 到 VM2 的报文添加租户识别流表
Switch# ovs-ofctl add-flow br0 "table=1,priority=1,actions=goto_table:2" -O openflow13	添加过滤流表
Switch# ovs-ofctl add-flow br0 "table=2,priority=65535,metadata=10,dl_dst=fa:16:3e:11:11:11,actions=set_tunnel:10,output:201" -O openflow13	为 VM2 到 VM1 的报文添加二层转发流表，做 tunnel encap
Switch# ovs-ofctl add-flow br0 "table=2,priority=65535,metadata=10,dl_dst=fa:16:3e:22:22:22,actions=output:1" -O openflow13	为 VM1 到 VM2 的报文添加二层转发流表，报文解封装后送到出接口 eth-0-1
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

验证配置

验证交换机上的 nvcloud 流表配置。

Switch2# ovs-ofctl dump-flows br0 -O openflow13

```

OFPST_FLOW reply (OF1.3) (xid=0x2):
  cookie=0x0, duration=33.744s, table=0, n_packets=0, n_bytes=0,
  priority=65535,in_port=1,dl_src=fa:16:3e:22:22:22
  actions=write_metadata:0xa,goto_table:1
  cookie=0x0, duration=27.345s, table=0, n_packets=0, n_bytes=0,
  priority=65535,tun_id=0xa actions=write_metadata:0xa,goto_table:1
  cookie=0x0, duration=20.864s, table=1, n_packets=0, n_bytes=0, priority=1
  actions=goto_table:2
  cookie=0x0, duration=7.094s, table=2, n_packets=?, n_bytes=?,
  priority=65535,metadata=0xa,dl_dst=fa:16:3e:22:22:22 actions=output:1

```

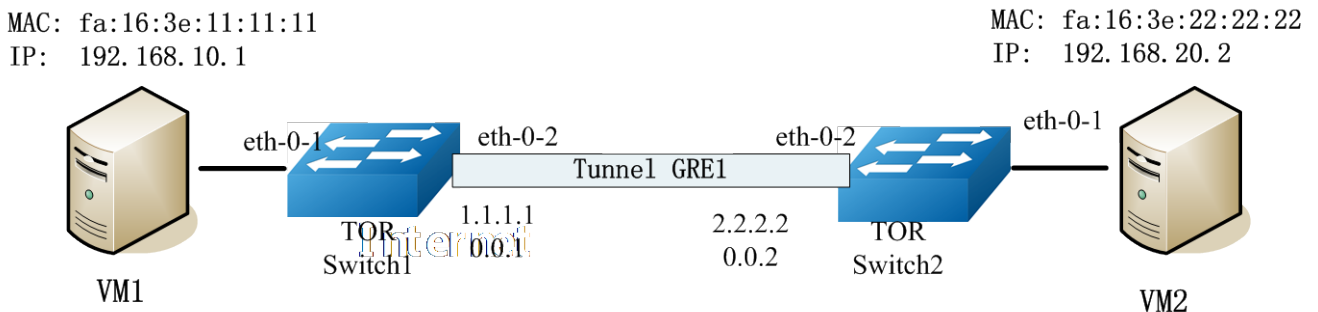
```
cookie=0x0, duration=15.471s, table=2, n_packets=?, n_bytes=?,  
priority=65535, metadata=0xa, dl_dst=fa:16:3e:11:11:11  
actions=set_tunnel:0xa, output:201
```

注意

如果用户需要 mac 学习功能，系统支持配置上 controller 的 miss flow，命令类似于：
ovs-ofctl add-flow br0 "table=2,priority=0,actions=controller" -O openflow13。

16.7 配置 L3 forward flow

以下示例介绍如何配置 l3forward 流表。假设 l3gateway 的 mac 地址为 fa:16:3e:22:22:00，
VM1 和 VM2 分别在不同子网。



Switch1

Switch(config)# tenant identify type port-mac	配置租户识别模式为 port-mac
Switch# ovs-vsctl add-port br0 gre1 -- set interface gre1 type=gre options:remote_ip=2.2.2.2 options:local_ip=1.1.1.1 options:bind_port=eth-0-2 options:nexthop_mac=0000.0000.0002	创建 gre 端口 gre1，本地 ip 为 1.1.1.1，远 端 IP 为 2.2.2.2，下一跳 mac 为 0.0.2，绑 定端口 eth-0-2
Switch# ovs-ofctl add-flow br0 "table=0,priority=65535,in_port=1,dl_src=f a:16:3e:11:11:11,actions=write_metadata:1 0,goto_table:1" -O openflow13	为 VM1 到 VM2 的报文添加租户识别流表

Switch# ovs-ofctl add-flow br0 "table=0,priority=65535,tun_id=10,actions= write_metadata:10,goto_table:1" -O openflow13	为 VM2 到 VM1 的报文添加租户识别流表
Switch# ovs-ofctl add-flow br0 "table=1,priority=1,actions=goto_table:2" -O openflow13	添加过滤流表
Switch# ovs-ofctl add-flow br0 "table=2,priority=65535,metadata=10,dl_dst= fa:16:3e:22:22:00,actions=goto_table:3" -O openflow13	添加 gateway 的 mac 地址，转到 table3 继续处理
Switch# ovs-ofctl add-flow br0 "table=3,priority=65535,metadata=10,ip,nw _dst=192.168.20.0/24,actions=strip_vlan,set _tunnel:10,output:201" -O openflow13	为 VM1 到 VM2 的报文添加三层转发流表，做 tunnel encap
Switch# ovs-ofctl add-flow br0 "table=3,priority=65535,metadata=10,ip,nw _dst=192.168.10.0/24,actions=mod_dl_src= fa:16:3e:22:22:00,mod_dl_dst=fa:16:3e:11: 11:11,output:1" -O openflow13	为 VM2 到 VM1 的报文添加三层转发流表，报文解封装后送到出接口 eth-0-1
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

验证配置

验证交换机上的 nvcloud 流表配置。

Switch1# ovs-ofctl dump-flows br0 -O openflow13

```

OFPST_FLOW reply (OF1.3) (xid=0x2):
  cookie=0x0, duration=72.053s, table=0, n_packets=0, n_bytes=0,
  priority=65535,in_port=1,dl_src=fa:16:3e:11:11:11
  actions=write_metadata:0xa,goto_table:1
  cookie=0x0, duration=64.008s, table=0, n_packets=0, n_bytes=0,
  priority=65535,tun_id=0xa actions=write_metadata:0xa,goto_table:1
  cookie=0x0, duration=39.983s, table=1, n_packets=0, n_bytes=0, priority=1
  actions=goto_table:2
  cookie=0x0, duration=32.859s, table=2, n_packets=?, n_bytes=?,
  priority=65535,metadata=0xa,dl_dst=fa:16:3e:22:22:00 actions=goto_table:3

```

```
cookie=0x0, duration=20.584s, table=3, n_packets=?, n_bytes=?,
priority=65535, ip, metadata=0xa, nw_dst=192.168.10.0/24
actions=mod_dl_src:fa:16:3e:22:22:00, mod_dl_dst:fa:16:3e:11:11:11, output:1
cookie=0x0, duration=26.1s, table=3, n_packets=?, n_bytes=?,
priority=65535, ip, metadata=0xa, nw_dst=192.168.20.0/24
actions=strip_vlan, set_tunnel:0xa, output:201
```

Switch2

Switch(config)# tenant identify type port-mac	配置租户识别模式为 port-mac
Switch# ovs-vsctl add-port br0 gre1 -- set interface gre1 type=gre options:remote_ip=1.1.1.1 options:local_ip=2.2.2.2 options:bind_port=eth-0-2 options:nexthop_mac=0000.0000.0001	创建 gre 端口 gre1, 本地 ip 为 2.2.2.2, 远端 IP 为 1.1.1.1, 下一跳 mac 为 0.0.1, 绑定端口 eth-0-2
Switch# ovs-ofctl add-flow br0 "table=0,priority=65535,in_port=1,dl_src=fa:16:3e:22:22:22,actions=write_metadata:10,goto_table:1" -O openflow13	为 VM2 到 VM1 的报文添加租户识别流表
Switch# ovs-ofctl add-flow br0 "table=0,priority=65535,tun_id=10,actions=write_metadata:10,goto_table:1" -O openflow13	为 VM1 到 VM2 的报文添加租户识别流表
Switch# ovs-ofctl add-flow br0 "table=1,priority=1,actions=goto_table:2" -O openflow13	添加过滤流表
Switch# ovs-ofctl add-flow br0 "table=2,priority=65535,metadata=10,dl_dst=fa:16:3e:22:22:00,actions=goto_table:3" -O openflow13	添加 gateway 的 mac 地址, 转到 table3 继续处理
Switch# ovs-ofctl add-flow br0 "table=3,priority=65535,metadata=10,ip,nw_dst=192.168.10.0/24,actions=strip_vlan,set_tunnel:10,output:201" -O openflow13	为 VM2 到 VM1 的报文添加三层转发流表, 做 tunnel encaps
Switch# ovs-ofctl add-flow br0 "table=3,priority=65535,metadata=10,ip,nw_dst=192.168.20.0/24,actions=mod_dl_src=fa:16:3e:22:22:00,mod_dl_dst=fa:16:3e:22:22:22,output:1" -O openflow13	为 VM1 到 VM2 的报文添加三层转发流表, 报文解封装后送到出接口 eth-0-1

```
Switch# ovs-ofctl dump-flows br0 -O openflow13
```

显示流表配置

验证配置

验证交换机上的 nvcloud 流表配置。

```
Switch2# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPPST_FLOW reply (OF1.3) (xid=0x2):
  cookie=0x0, duration=49.308s, table=0, n_packets=0, n_bytes=0,
  priority=65535,in_port=1,dl_src=fa:16:3e:22:22:22
  actions=write_metadata:0xa,goto_table:1
  cookie=0x0, duration=42.928s, table=0, n_packets=0, n_bytes=0,
  priority=65535,tun_id=0xa actions=write_metadata:0xa,goto_table:1
  cookie=0x0, duration=35.96s, table=1, n_packets=0, n_bytes=0, priority=1
  actions=goto_table:2
  cookie=0x0, duration=28.258s, table=2, n_packets=?, n_bytes=?,
  priority=65535,metadata=0xa,dl_dst=fa:16:3e:22:22:00 actions=goto_table:3
  cookie=0x0, duration=21.098s, table=3, n_packets=?, n_bytes=?,
  priority=65535,ip,metadata=0xa,nw_dst=192.168.10.0/24
  actions=strip_vlan,set_tunnel:0xa,output:201
  cookie=0x0, duration=15.246s, table=3, n_packets=?, n_bytes=?,
  priority=65535,ip,metadata=0xa,nw_dst=192.168.20.0/24
  actions=mod_dl_src:fa:16:3e:22:22:00,mod_dl_dst:fa:16:3e:22:22:22,output:1
```

注意

三层转发表不支持报文上 controller。用户需要配置一条缺省 entry 将报文送到网关，命令类似于： ovs-ofctl add-flow br0 "table=3,priority=0,actions=output:3" -O openflow13。其中 port3 连接网关。

17 配置 Multi-table 模式

17.1 概述

Multi-table 流表模式是盛科为多级流表应用定制的专用流表模式，此模式支持两张流表，其中 table0 支持 500 条流表和 6000 条 tunnel 流表，table1 支持 1000 条 wildcard 匹配的流表。

Profile Type	Table 0		Table 1
	Wildcard match flow	Tunnel flow	Wildcard match flow
流表数量	500	6000	1000
Wildcard 匹配字段	IN_PORT (PHYSICAL) MAC_SA/MAC_DA VLAN_ID/VLAN_PCP ETHER_TYPE ICMP TYPE/ICMP CODE IPSA/IPDA/IP_DSCP L3_PROTOCOL TCP_SRC_PORT/TCP_DST_PORT UDP_SRC_PORT/UDP_DST_PORT	不支持	METADATA MAC_SA/MAC_DA VLAN_ID/VLAN_PCP ETHER_TYPE ICMP TYPE/ICMP CODE ARP_OP/ARP_SPA/ARP_TPA IPSA/IPDA/IP_DSCP L3_PROTOCOL TCP_SRC_PORT/TCP_DST_PORT UDP_SRC_PORT/UDP_DST_PORT

精确匹配 字段		IN_PORT (TUNNEL) TUNNEL_ID(optional)	
流表行为	OUTPUT SET_FIELD(MAC_SA/MAC_DA/ VLAN_VID/VLAN_PCP/IPV4_DA/TCP_DST_PORT/UDP_DST_PORT/TUNNEL_ID) PUSH_VLAN/POP_VLAN DEC_IP_TTL	OUTPUT SET_FIELD(MAC_SA/MAC_DA/ VLAN_VID/VLAN_PCP/IPV4_DA/TCP_DST_PORT/UDP_DST_PORT/TUNNEL_ID) PUSH_VLAN/POP_VLAN DEC_IP_TTL	OUTPUT SET_FIELD(MAC_SA/MAC_DA/ VLAN_VID/VLAN_PCP/IPV4_DA/TCP_DST_PORT/UDP_DST_PORT/TUNNEL_ID) PUSH_VLAN/POP_VLAN DEC_IP_TTL
Write Metadata	支持	支持	不支持
Goto table	支持	支持	不支持
Group	支持		
MPLS	不支持		
GRE	支持		
QinQ	支持		
Meter	支持		
Queue	支持		
sFlow	支持		

17.2 参考资料

- OpenFlow 1.3.1 specification
- Open vSwitch

17.3 配置 multi-table 流表模式

以下示例介绍如何切换交换机到 multi-table 流表模式。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# stm prefer multi-table	切换到 multi-table 流表模式
Switch(config)# end	退出到特权视图
Switch# show stm prefer	显示当前和下一次重启使用的流表模式
Switch# reboot	重启系统以便新的流表模式生效

验证配置

验证交换机上的 multi-table 流表模式配置（尚未重启）。

Switch1# show stm prefer

```
Current profile is : multi-table
Next profile is   : multi-table
```

17.4 配置 multi-table 的 wildcard match 流表

以下示例介绍如何在 multi-table 模式下配置 wildcard match 流表。

Switch

Switch# ovs-ofctl add-flow br0 table=0,in_port=1,actions=write_metadata:1 00,goto_table:1 -O openflow13	为 table0 添加流，action 为 write_metadata 和 goto_table
---	--

Switch# ovs-ofctl add-flow br0 table=1,metadata=100,actions=output:2 -O openflow13	为 table1 添加流, 匹配 metadata 100, action 为 output 2
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

验证配置

验证交换机上的 multi-table 流表配置。

```
Switch# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPT_FLOW reply (OF1.3) (xid=0x2):
  cookie=0x0, duration=9.545s, table=0, n_packets=0, n_bytes=0, in_port=1 流表行为
  =write_metadata:0x64,goto_table:1
  cookie=0x0, duration=5.699s, table=1, n_packets=0, n_bytes=0, metadata=0x64 act
  ions=output:2
```

17.5 配置 multi-table 的 tunnel 流表

以下示例介绍如何在 multi-table 模式下配置 tunnel 流表。

Switch

Switch# ovs-vsctl add-port br0 gre1 -- set interface gre1 type=gre options:remote_ip=5.5.5.5 options:local_ip=6.6.6.6 options:bind_port=eth-0-1 options:nexthop_mac=0.0.1	创建 gre 端口 gre1, 本地 ip 为 6.6.6.6, 远 端 IP 为 5.5.5.5, 下一跳 mac 为 0.0.1, 绑 定端口 eth-0-1
Switch# ovs-ofctl add-flow br0 table=0,gre,actions=gre_decap -O openflow13	为 table0 添加流表, 解封装 gre 报文
Switch# ovs-ofctl add-flow br0 table=0,priority=0xffff,in_port=201,actions =write_metadata:100,goto_table:1 -O openflow13	为 table0 添加流表, 匹配 tunnel 201 收到 的报文, 更新 metadata 并送到 table1 中

Switch# ovs-ofctl add-flow br0 table=1,metadata=100,actions=output:2 -O openflow13	为 table1 添加流表，匹配 metadata 100 并 送到 eth-0-2
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

验证配置

验证交换机上的多级流表配置。

```
Switch# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPST_FLOW reply (OF1.3) (xid=0x2):  
  cookie=0x0, duration=11.017s, table=0, n_packets=0, n_bytes=0, priority=65535,i  
n_port=201 actions=write_metadata:0x64,goto_table:1  
  cookie=0x0, duration=16.736s, table=0, n_packets=0, n_bytes=0, gre actions=GRE_  
DECAP  
  cookie=0x0, duration=6.097s, table=1, n_packets=0, n_bytes=0, metadata=0x64 act  
ions=output:2
```

18 配置 IPv6 模式

18.1 概述

IPv6 流表模式是盛科为 IPv6 应用定制的专用流表模式，此模式支持 500 条 wildcard 流表。

Profile Type	Table	
	mac/arp/ipv4 wildcard match flow	ipv6 wildcard match flow
流表数量	500	
Wildcard 匹配字段	IN_PORT MAC_SA/MAC_DA VLAN_ID/VLAN_PCP ETHER_TYPE ICMP TYPE/ICMP CODE IPSA/IPDA/IP_DSCP L3_PROTOCOL TCP_SRC_PORT/TCP_DST_PORT UDP_SRC_PORT/UDP_DST_PORT ARP opcode/ARP SPA/TPA	IN_PORT MAC_SA/MAC_DA VLAN_ID/VLAN_PCP ETHER_TYPE ICMP TYPE/ICMP CODE IPSA/IPDA/IP_DSCP L3_PROTOCOL TCP_SRC_PORT/TCP_DST_PORT UDP_SRC_PORT/UDP_DST_PORT
精确匹配字段		

流表行为	OUTPUT SET_FIELD(MAC_SA/MAC_DA/ VLAN_VID/VLAN_PCP/IPV4_DA /TCP_DST_PORT/UDP_DST_POR T/TUNNEL_ID) PUSH_VLAN/POP_VLAN DEC_IP_TTL	OUTPUT SET_FIELD(MAC_SA/MAC_DA/ VLAN_VID/VLAN_PCP /TUNNEL_ID) PUSH_VLAN/POP_VLAN
Group	支持	
MPLS	支持	
GRE	支持	
QinQ	支持	
Meter	支持	
Queue	支持	
sFlow	支持	

18.2 参考资料

- OpenFlow 1.3.1 specification
- Open vSwitch

18.3 Configure ipv6 profile

以下示例介绍如何切换交换机到 ipv6 流表模式。

Switch

Switch# configure terminal	进入配置视图
Switch(config)# stm prefer ipv6	切换到 ipv6 流表模式
Switch(config)# end	退出到特权视图

Switch# show stm prefer	显示当前和下一次重启使用的流表模式
Switch# reboot	重启系统以便新的流表模式生效

验证配置

验证交换机上的 ipv6 流表模式配置（尚未重启）。

```
Switch1# show stm prefer
```

```
Current profile is : ipv6
Next profile is   : ipv6
```

18.4 Add ipv6 wildcard match flows

以下示例介绍如何在 ipv6 模式下配置 wildcard 流表。

Switch

Switch# ovs-ofctl add-flow br0 ipv6,ipv6_label=100,nw_proto=58,icmp_code=1,icmp_type=2,actions=output:2 -O openflow13	添加一条流，匹配 IPv6 字段
Switch# ovs-ofctl dump-flows br0 -O openflow13	显示流表配置

验证配置

验证交换机上的 ipv6 流表配置。

```
Switch# ovs-ofctl dump-flows br0 -O openflow13
```

```
OFPST_FLOW reply (OF1.3) (xid=0x2):
 cookie=0x0, duration=42.267s, table=0, n_packets=0, n_bytes=0, icmp6,ipv6_label=0x000064,icmp_type=2,icmp_code=1 actions=output:2
```

19 配置 LACP

19.1 概述

V350/V150 支持在 Bond 端口上使能 LACP 协议。

在直连的两个交换机间通过交换 LACP 报文，可以实现动态链路聚合，并提供较好的灵活性和可控制性。

19.2 配置 LACP

在下面的配置示例中，在 V350/V150 交换机上创建 Bond 端口 bond1，其成员口为物理端口 1 和端口 2。

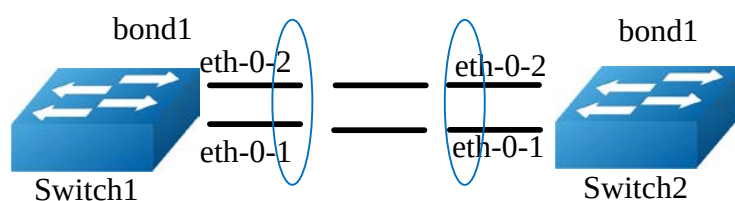


Figure 19-1 : LACP 拓扑

Switch1

Switch# ovs-vsctl del-port br0 eth-0-1	从 br0 中删除物理端口 1
Switch# ovs-vsctl del-port br0 eth-0-2	从 br0 中删除物理端口 2
Switch# ovs-vsctl add-bond br0 bond1 eth-0-1 eth-0-2 lacp=active bond_mode=balance-slb -- set interface eth-0-1 type=switch -- set interface eth-0-2 type=switch	创建负载分担类型的 bond1 端口，成员口为物理端口 1 和物理端口 2，并使能 LACP 协议

Switch2

Switch# ovs-vsctl del-port br0 eth-0-1	从 br0 中删除物理端口 1
Switch# ovs-vsctl del-port br0 eth-0-2	从 br0 中删除物理端口 2
Switch# ovs-vsctl add-bond br0 bond1 eth-0-1 eth-0-2 lacp=active bond_mode=balance-slb -- set interface eth-0-1 type=switch -- set interface eth-0-2 type=switch	创建负载分担类型的 bond1 端口，成员口为物理端口 1 和物理端口 2，并使能 LACP 协议

配置验证

Step 1 验证交换机上的 Bond 配置。

Switch1# **show interface bond brief**

```
The Maximum of bond port is 16, currently 1 bond ports is valid

---- bond1 ----
bond_mode   : balance-slb
bond_id     : 1
bond_ofport : 401
gre_ref     : 0
lacp_status : negotiated

slave eth-0-1: enabled

slave eth-0-2: enabled
```

Switch1# **ovs-appctl dump-lacp**

```
---- bond1 ----
  status: active negotiated
  sys_id: 06:a6:a7:67:38:00
  sys_priority: 65534
  aggregation key: 2
  lacp_time: slow

slave: eth-0-1: current attached
  port_id: 1
  port_priority: 65535
  may_enable: true

  actor sys_id: 06:a6:a7:67:38:00
  actor sys_priority: 65534
```

```

actor port_id: 1
actor port_priority: 65535
actor key: 2
actor state: activity aggregation synchronized collecting distributing

partner sys_id: 6e:77:c4:b0:ae:00
partner sys_priority: 65534
partner port_id: 1
partner port_priority: 65535
partner key: 2
partner state: activity aggregation synchronized collecting distributing

slave: eth-0-2: current attached
port_id: 2
port_priority: 65535
may_enable: true

actor sys_id: 06:a6:a7:67:38:00
actor sys_priority: 65534
actor port_id: 2
actor port_priority: 65535
actor key: 2
actor state: activity aggregation synchronized collecting distributing

partner sys_id: 6e:77:c4:b0:ae:00
partner sys_priority: 65534
partner port_id: 2
partner port_priority: 65535
partner key: 2
partner state: activity aggregation synchronized collecting distributing

```

19.3 配置 LACP 参数

Switch

Switch# ovs-vsctl set port bond1 lacp-system-id 00:1E:08:08:FD:83	配置 LACP 系统 id
Switch# ovs-vsctl set port bond1 lacp-system-priority 65535	配置 LACP 系统优先级
Switch# ovs-vsctl set port bond1 lacp-time fast	配置 LACP 发送速率
Switch# ovs-vsctl set port bond1 lacp-status passive	配置 LACP 状态

Switch# ovs-vsctl set interface eth-0-1 lacp-port-id 65535	配置 LACP 端口 id
Switch# ovs-vsctl set interface eth-0-1 lacp-port-priority 1000	配置 LACP 端口优先级
Switch# ovs-vsctl set interface eth-0-1 lacp-aggregation-key 100	配置 LACP 聚合 key

配置验证

Step 1 验证交换机上的 Bond 配置。

Switch1# **ovs-appctl dump-lacp**

```

---- bond1 ----
  status: passive negotiated
  sys_id: 00:1e:08:08:fd:83
  sys_priority: 65535
  aggregation key: 100
  lacp_time: fast

slave: eth-0-10: current attached
  port_id: 10
  port_priority: 65535
  may_enable: true

  actor sys_id: 00:1e:08:08:fd:83
  actor sys_priority: 65535
  actor port_id: 10
  actor port_priority: 65535
  actor key: 100
  actor state: timeout aggregation synchronized collecting distributing

  partner sys_id: 6e:77:c4:b0:ae:00
  partner sys_priority: 65534
  partner port_id: 10
  partner port_priority: 65535
  partner key: 9
  partner state: activity aggregation synchronized collecting distributing

slave: eth-0-9: current attached
  port_id: 65535
  port_priority: 1000
  may_enable: true

  actor sys_id: 00:1e:08:08:fd:83
  actor sys_priority: 65535
  actor port_id: 65535
  actor port_priority: 1000
  actor key: 100

```

```
actor state: timeout aggregation synchronized collecting distributing

partner sys_id: 6e:77:c4:b0:ae:00
partner sys_priority: 65534
partner port_id: 9
partner port_priority: 65535
partner key: 9
partner state: activity aggregation synchronized collecting distributing
```

20 配置启动时加载 flows/groups/meters

20.1 概述

V350/V150 支持保存当前 flows, groups, meters 等的配置到启动配置文件中, 系统重启时, 会自动加载这些配置。

注意: write-startup-flows/groups/meters 等命令会以系统配置的最高 OpenFlow 版本要求的格式保存配置到 flash 下的 init_flow/init_group/init_meter 等文件中, 系统重启加载这些配置文件时, 也会按照系统配置的最高 OpenFlow 版本的格式来恢复这些配置 (即 ovs-ofctl 命令会使用 -O 选项来指定版本号)。

20.2 配置启动时加载 flows

以下示例介绍如何保存或清除启动加载 flows。

Switch

Switch# ovs-ofctl write-startup-flows br0	保存系统当前 flows 配置到文件 “init_flow”
Switch# ovs-ofctl clear-startup-flows br0	清除配置文件 “init_flow” 中的配置

验证配置

验证交换机上 “init_flow” 文件中的配置

```
Switch1# ovs-ofctl write-startup-flows br0
```

```
Switch1# more init_flow
```

```
cookie=0x0, table=0, ip,in_port=2,nw_src=192.168.1.2,nw_dst=192.168.1.1 流表行为
```

```
=output:1
cookie=0x0, table=0, arp,in_port=1,arp_spa=192.168.1.1,arp_tpa=192.168.1.2 acti
ons=output:2
cookie=0x0, table=0, arp,in_port=2,arp_spa=192.168.1.2,arp_tpa=192.168.1.1 acti
ons=output:1
cookie=0x0, table=0, ip,in_port=1,nw_src=192.168.1.1,nw_dst=192.168.1.2 流表行为
=output:2
```

20.3 配置启动时加载 groups

以下示例介绍如何保存或清除启动加载 groups。

Switch

Switch# ovs-ofctl write-startup-groups br0	保存系统当前 groups 配置到文件 “init_group”
Switch# ovs-ofctl clear-startup-groups br0	清除配置文件 “init_group” 中的配置

验证配置

验证交换机上 “init_group “文件中的配置

```
Switch1# ovs-ofctl write-startup-groups br0
```

```
Switch1# more init_group
```

```
group_id=1, type=ALL, bucket=output:1, bucket=output:2
```

20.4 配置启动时加载 meters

以下示例介绍如何保存或清除启动加载 meters。

Switch

Switch# ovs-ofctl write-startup-meters br0	保存系统当前 meters 配置到文件 “init_meter”
Switch# ovs-ofctl clear-startup- meters br0	清除配置文件 “init_meter” 中的配置

验证配置

验证交换机上 “init_meter “文件中的配置

```
Switch1# ovs-ofctl write-startup- meters br0
```

```
Switch1# more init_ meter
```

```
meter_id=1, flags=KBPS-BURST-STATS, band_type=DROP, rate=10000, burst=128
```

21 附录

21.1 附录 A

流表描述

流描述应该采用规范的格式，比如，一个流只有在同时指定了二层协议类型的情况下才能指定三层字段，同理，只有在指定三层协议类型的情况下才能指定四层字段。这些描述由一系列的 *field=value* 格式组成，通过空格或逗号分割。

以下是系统支持的字段：

- **in_port = port_number**
匹配 OpenFlow 端口 port_number。
- **dl_src = xx:xx:xx:xx:xx:xx**
Ethernet source address
匹配源 MAC 地址。
- **dl_dst = xx:xx:xx:xx:xx:xx [/ xx:xx:xx:xx:xx:xx]**
匹配目的 MAC 地址。
- **dl_type = ethertype**
匹配以太网协议类型 **ethertype**，取值 0 至 65535。支持十进制和十六进制格式（比如：0x0806 表示 ARP）。
- **dl_vlan_pcp = priority**
Matches IEEE 802.1q Priority Code Point (PCP) priority
- **dl_vlan = vlan**
Matches IEEE 802.1q Virtual LAN tag vlan
匹配 IEEE802.1q 的虚拟局域网 *vlan*；0xffff 表示匹配未带 VLAN tag 的报文，否则可指定 1 至 4094 间的数作为 VLAN ID。

- **vlan_tci = tci**

Matches modified VLAN TCI.

- **nw_src = ip[/netmask]**

- **nw_dst = ip[/netmask]**

匹配 IPv4 源/目的地址

如果 dl_type 设置为 0x0800（或缩写，例如：ip 或 tcp），匹配 IPv4 源（或目的）IP 地址，可选项 *netmask* 允许设置可匹配的 IPv4 前缀长度。

如果 dl_type 设置为 0x0806 或设置为 arp，相应的，匹配 arp 报文中的 arp_spa 或 arp_tpa 字段。

如果 dl_type 被忽略掉，或设置为 0x0800 和 0x0806 以外的值，则 nw_src 和 nw_dst 会被忽略。

- **nw_proto = proto**

如果指定 ip 报文或 dl_type=0x0800，匹配 ip 协议类型 *proto*，取值范围 0 至 255（比如：1 匹配 ICMP，6 匹配 TCP）。

如果指定 ipv6 或 dl_type=0x86dd，匹配 ipv6 报文头中协议类型 *proto*，取值范围 0 至 255（比如：58 匹配 ICMPv6，6 匹配 TCP）。

如果指定 arp 或 dl_type=0x0806，匹配 ARP 报文中操作码中低 8 位，大于 255 当做 0 处理。

如果 dl_type 被忽略掉，或设置为 0x0800，0x0806 和 0x86dd 以外的值，则 nw_proto 会被忽略。

- **nw_tos = tos**

匹配 IP 报文中 Tos/DSCP 字段或 IPv6 中 tos 字段，取值范围 0 至 255.注意：低 2 位预留未使用。

如果 dl_type 被忽略掉，或设置为 0x0800，0x0806 和 0x86dd 以外的值，则 nw_tos 会被忽略。

- **tp_src = port**

- **tp_dst = port**

如果 nw_proto 指定为 TCP 或 UDP，tp_src 和 tp_dst 匹配 UDP 或 TCP 的源或目的端口，取值范围 0 至 65535。

如果 nw_proto 为其他值，该设置会被忽略。

- **icmp_type = type**

- **icmp_code = code**

如果 nw_proto 指定为 ICMP 或 ICMPv6，type 匹配 ICMP 类型，code 匹配 ICMP 的 code，取值范围 0 至 65535。

如果 `dl_type` 和 `nw_proto` 为其他值，该设置会被忽略。

- **idle_timeout = seconds**

设定空闲老化时间（以秒为单位），设置为 0 时表示不老化。

- **hard_timeout = seconds**

设定强制老化时间（以秒为单位），设置为 0 时表示不老化。

流表语法支持以下缩写。

ip	<code>dl_type = 0x0800</code>
tcp	<code>dl_type = 0x0800, nw_proto=6</code>
udp	<code>dl_type = 0x0800, nw_proto=17</code>
arp	<code>0x0806</code>
icmp	<code>dl_type = 0x0800, nw_proto=1</code>
ipv6	<code>dl_type=0x86dd</code>
tcp6	<code>dl_type=0x86dd,nw_proto=6</code>
udp6	<code>dl_type=0x86dd,nw_proto=17</code>
icmp6	<code>dl_type=0x86dd,nw_proto=58</code>

- **ipv6_src=ipv6[/netmask]**
- **ipv6_dst=ipv6[/netmask]**

如果 `dl_type` 取值 0x86dd（缩写：ipv6），匹配 IPv6 源（或目的）地址。

- **tun_id=tunnel-id**

匹配 GRE 报文中的 GRE key 字段。



更多信息，请参考 open vswitch 手册：ovsman-ovs-ofctl.pdf，第三页。

流表行为

系统支持将流表绑定到 meter 或 queue 上，对不同类型的流量进行调度。

- **meter:meter_id**
将流绑定到一个 meter 上。该 action 需要放到所有其他 action 之前。
- **group:group_id**
将流绑定到一个 group 上。该 action 不能和其他 action 共存。
- **set_queue:queue_id**
将流绑定到一个 queue 上。该 action 需要放到 meter（如果存在）之后，其他 action 之前。

指定在匹配的 flow 上生效的 action 时，可以是一个十进制的端口号，该端口是匹配到 flow 的报文的出端口，或者是以下关键字：

- **drop**
丢弃报文，不能与其他 action 混用。
- **controller**
发送 “packet in “ 报文到控制器
- **output**
指定报文的出接口。
- **all**
报文会发送到除入接口之外的所有接口。
- **in_port**
指定报文从接口端口发送出去。

如果 action 中包含 output，那么关键字 “set-field” 可以用来修改匹配到该 flow 的报文里的某些字段。不同 set-field 之间以逗号分割，同一组 set-field 的格式为 field:value。

支持 set-field 语法的字段如下：

- **set_field: mac_address->eth_src**
设置新的源 MAC 地址。
- **set_field: mac_address->eth_dst**
设置新的目的 MAC 地址。
- **set_field: vlan_id->vlan_vid**

修改报文中 VLAN id, VLAN tag 可能会被修改或者增加上去, 如果需要增加一个 VLAN tag, priority 默认值为 0。

- **set_field:** ip->nw_dst
设置新的 IPv4 目的地址
- **set_field:** port->tp_dst
设置新的 TCP 目的端口 **set_field:** port->tcp_dst
设置新的 UDP 目的端口 **set_field:** port->udp_dst
- **set_field:** tunnel_id->tun_id
设置 tunnel id
- **push_vlan:** ethertype
修改报文中 VLAN 的 priority。如果报文中没有 VLAN tag, 则 action 会被忽略。
- **strip_vlan**
剥去报文中外层 VLAN tag。

- **pop_vlan**

同命令 “strip_vlan” 剥去报文中外层 VLAN tag。

21.2 附录 B

SSL 配置

本节介绍 Open vSwitch SSL 认证所采用的公共密钥基础架构（PKI）。

如果要使用 SSL 进行连接，每个 Open vSwitch 必须有唯一的密钥对（公私/私钥）和对公钥签名的数字证书。通常情况下，Open vSwitch 会生成自己的密钥对。获取证书的两种常用方法：

- **自签名证书：**Open vSwitch 使用自己的私钥签名证书，在这种情况下，因为没有中央的认证，每个交换机都需要独立地被控制器所授权。这是 NOX 控制器目前支持的唯一的 PKI 模型。
- **交换机证书认证：**证书认证机构（即交换机 CA）为每个 Open vSwitch 公钥签名。控制器会验证发起连接的交换的数字证书是否被该 CA 所签名。这是 Open vSwitch 自带的简单 OpenFlow 控制器支持的唯一 PKI 模型。

每一个 Open vSwitch 交换机必须有一个用于签名 OpenFlow 控制器密钥的 CA 证书的副本。通常情况下，同一 CA 证书会被安装到其管理域内的所有交换机上。

以下是交换机获取控制器 CA 证书的两种常用方法：

- 1、通过某种安全方式手动拷贝控制器的 CA 证书到交换机。例如：使用 USB 拷贝，通过网络命令 “scp” 或需要提供用户名/密码的 FTP/HTTP 等方式。
- 2、打开 Open vSwitch 的 “bootstrap” 模式，该模式下，Open vSwitch 交换机会在第一次连接 OpenFlow 控制器时获取 CA 证书并保存到本地。此后，交换机只连接到由该 CA 证书签名的交换机。

建立公共密钥基础设施

Open vSwitch 交换机可以利用已存在的公共密钥基础设施。如果你已经有了 PKI，可以转到下一节，否则，`ovs-pki` 命令可以帮你创建一个初始的 PKI 结构：

% ovs-pki init

创建并使用一个新的PKI路径。路径的默认位置取决于Open vSwitch的配置（详情请参考“ovs-pki --help”命令中的--dir选项）。

PKI路径下有两个重要的子目录。控制器CA目录包含控制器的CA文件，包括：

- **cacert.pem**: 用于认证控制器的根证书。每个 Open vSwitch 交换机必须包含该文件的一个拷贝。用来认证合法的控制器。
- **private/cakey.pem**: 私钥签名的 key，用来认证控制器。该文件需要保密存放。交换机和控制器不需要该文件的副本。

交换机的CA目录存放交换的CA文件。类似的，控制器的CA目录也包括：

- **Cacert.pem**: 用于认证交换机的根证书。控制器使用此证书认证合法的交换机。
- **Private/cakey.pem**: 私钥签名的 key，用于认证交换机。该文件需要保密存放。交换机和控制器不需要该文件的副本。

创建PKI的初始结构后，可以通过ovs-pki命令为交换机和控制器创建密钥和数字证书。请参考ovs-pki(8)获取详细信息。以下为使用示例：

- 生成控制器密钥

生成控制器的私钥 **ctl-privkey.pem** 和数字证书 **ctl-cert.pem**，请运行以下命令：

```
% ovs-pki req+sign ctl controller
```

私钥 **ctl-privkey.pem** 和数字证书 **ctl-cert.pem** 需要拷贝到控制器。并且需要通过 **--private-key** 和 **--certificate** 选项指定上面生成的文件。

注意：请确保 **ctl-cert.pem** 文件没有存在其他不必要的备份，因为它可以被恶意的用于冒充控制器。

- 生成交换机的自签名密钥和数字证书

自签名数字证书（见“OpenFlow 的 SSL”一节）是控制器支持的可接受的证书的一种，

1、在交换机上运行以下命令：

```
% ovs-pki self-sign sc
```

（该命令不需要任何“ovs-pki init”生成的 PKI 文件的副本，并且你不应当把其复制到 switch，因为它们有的内容是需要保密的）

“ovs-pki self-sign”命令会生成以下文件：

- Sc-privkey.pem，交换机私钥。需要保密存在。通常情况下，不需要拷贝到交换机以为的地方。
- Sc-cert.pem，交换机数字证书。由交换机私钥签名。数字证书没有保密要求。

2、可选项，从存放 OpenFlow 的 PKI 结构的机器上拷贝控制器的 CA/ cacert.pem 文件。（否则，在下一步配置中，你必须选择使用 CA 数字证书引导）

3、配置 Open vSwitch 交换使用密钥和数字证书（参考下面“配置 SSL”一节）

- 使用 PKI 生成交换机密钥和数字证书（简单）

如果你使用的是 PKI 模式（参考上面“OpenFlow 的 SSL”一节），这种交换机生成密钥的方法比下面介绍的替代方案相对容易一点，但它相对的也缺少安全性，主要是因为它需要从生成 PKI 文件的机器上复制敏感的私钥到交换机。

1、在 PKI 结构的机器上运行以下命令：

```
% ovs-pki req+sign sc switch
```

该命令会生成如下文件：

- sc-privkey.pem，交换机私钥。为了安全性，该文件需要保密存放。
- sc-cert.pem，交换机数字证书。数字证书没有保密要求。

2、拷贝 sc-privkey.pem 和 sc-cert.pem 以及控制器的 CA/cacert.pem 文件到交换机

3、删除生成 PKI 文件的机器上保存的 sc-privkey.pem 和 sc-cert.pem 文件以及将此文件拷贝到交换机的中间过程中可能产生的文件副本。这些文件可以用来恶意伪装交换机，所以需要删除以提高安全性。

（请不要删除控制器的 CA/cacert.pem 文件，它不具有保密性而且你在配置其他交换机时有可能还会用到这些文件）

4、配置 Open vSwitch 交换使用密钥和数字证书（参考下面“配置 SSL”一节）

- 使用 PKI 生成交换机密钥和数字证书(安全)

如果你使用的是 PKI 模式（参考上面“OpenFlow 的 SSL”一节），相对于上面介绍的方法，下面的方法要多一些步骤，但是此方法不涉及私钥拷贝，因此，相对来说更安全一些。

在Open vSwitch交换机上运行以下命令：

```
% ovs-pki req sc switch
```

(该命令不需要任何“ovs-pki init”命令生成的文件，所以你也无需拷贝任何文件到交换机上，因为有些文件是需要保密存放的)

"ovs-pki req"命令会生成以下文件：

- sc-privkey.pem, 交换机私钥。安全起见，文件需要保密存放。一般情况下不需要拷贝到交换机以外的地方。
- sc-req.pem, 交换机的"certificate request"（证书签证请求），本质上是交换机的公钥，无保密要求。
- MD5值，显示在屏幕上的指纹值。

记下指纹并拷贝 sc-req.pem 到 PKI 结构所在机器。并运行如下命令

```
% ovs-pki sign sc switch
```

此命令会输出 MD5(fingerprint)到屏幕并请求用户确认。请检查该值是否和你记录的 MD5 值一致。

"ovs-pki sign"命令创建名为 sc-cert.pem 文件，该为为交换机的数字证书。无保密要求。

1. 拷贝生成的 sc-cert.pem 文件以及控制器的 CA/cacert.pem 文件到 Open vSwitch 交换机。
拷贝完成后可以删除 PKI 机器上的 sc-cert.pem 文件。（请不要删除控制器的 CA/cacert.pem 文件，它不具有保密性而且你在配置其他交换机时有可能还会用到这些文件）。
2. 配置 Open vSwitch 交换使用密钥和数字证书（参考下面“配置 SSL”一节）

A.1.1 对 SSL 配置的支持

SSL配置需要三个文件。前两个文件对每个交换机都是唯一的。如果你使用了上面的PKI命令创建的结构，则这两个文件名分别为sc-privkey.pem 和 sc-cert.pem。

- 私钥，包含一半的 RSA 或 DSA 密钥的信息。

为安全起见，这个文件可由 Open vSwitch 交换机本身生成，或在其他地方生成并拷贝到交换机上。

私钥内容需要安全保存，请勿外泄。

- 证书文件，证书无保密要求

第三个配置文件一般来说在所有由同一个控制器控制的交换机上都是一样的。如果你使用了上面的PKI命令创建的结构，则这个文件名为cacert.pem:

- 验证控制器的根文件。

如果您已经创建了上述文件，通过下面“set-ssl”命令来对系统进行配置，例如：

```
ovs-vsctl set-ssl /etc/openvswitch/sc-privkey.pem /etc/openvswitch/sc-cert.pem  
/etc/openvswitch/cacert.pem
```

注意：在 centec 系统中，可以使用“**openflow set ssl private-key sc-privkey.pem certificate sc-cert.pem ca-cert cacert.pem**”代替

请确保上述输入的文件名称正确，如果你使用了其他文件名，请使用绝对路径名（以“/”开头的路径），因为 ovs-vswitchd 进程的工作路径可能和你运行 ovs-vsctl 命令的路径不一致。

如果你使用了自签名的证书，则不需要从运行 PKI 的机器上拷贝控制器的 ca/cacert.pem 文件到交换机，这种情况下，您需要用到—bootstrap 选项，

例如：

```
ovs-vsctl -- --bootstrap set-ssl /etc/openvswitch/sc-privkey.pem  
/etc/openvswitch/sc-cert.pem /etc/openvswitch/cacert.pem
```

（注意：在 centec 系统中，可以使用“**openflow set ssl private-key sc-privkey.pem certificate sc-cert.pem ca-cert cacert.pem bootstrap**”代替）

当你添加了所有这些配置之后，你应该配置数据库的连接方法为“ssl:”，“tcp:”连接方式仍然是可以配置的，但为安全起见，你应该选择“ssl:”连接。

和大多数 Open vSwitch 配置不同，SSL 配置只在 ovs-vswitchd 进程启动时读取一次，如果在进程运行过程中，改动了 SSL 配置，若使新的配置生效，必须重启 ovs-vswitchd 进程。